

Jonas Gomes de Moraes
Luis Enrique del Llano Risco

Implementação e Avaliação de uma Solução Criptográfica para Mobilidade Segura em Eleições Brasileiras

São Paulo, SP

2025

Jonas Gomes de Moraes
Luis Enrique del Llano Risco

Implementação e Avaliação de uma Solução Criptográfica para Mobilidade Segura em Eleições Brasileiras

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Marcos Antonio Simplicio Junior

Coorientador: Leonardo Toshinobu Kimura

São Paulo, SP

2025

Resumo

A mobilidade em eleições favorece a diminuição da abstenção eleitoral. Dessa forma, o presente trabalho explora a viabilidade de soluções de software para mobilidade em eleições brasileiras. Nesse cenário, foram desenvolvidos uma solução criptográfica para o tratamento de possíveis votos duplicados por meio de *mix networks* e um protótipo de aplicativo para a seleção do local de votação de um eleitor. A solução criptográfica possibilita o tratamento de um grande número de votos em tempo hábil, e o protótipo de aplicativo executa um fluxo completo de escolha de local de votação e posterior autenticação do eleitor nesse local. Assim, ambas as soluções se mostraram viáveis.

Palavras-chave: mobilidade em eleições. *mix networks*. *software* eleitoral.

Abstract

Mobility in elections helps reduce voter abstention. Accordingly, this work explores the feasibility of software solutions aimed at enabling mobility in Brazilian elections. In this context, a cryptographic solution was developed to handle potential duplicate votes through the use of mix networks, as well as a prototype application that allows voters to select their polling location. The cryptographic solution makes it possible to process a large number of votes in a timely manner, and the prototype application performs a complete workflow for choosing a polling location and subsequently authenticating the voter at that location. Thus, both solutions were shown to be viable.

Keywords: mobility in elections. mix networks. electoral software.

Lista de ilustrações

Figura 1	–	Esquema simplificado de uma <i>mix network</i>	12
Figura 2	–	Esquema simplificado de uma <i>mix network</i> de recifração	13
Figura 3	–	Panorama da solução de mobilidade estudada baseada na remoção de votos duplos	18
Figura 4	–	Tela do setup inicial do frontend em linha de comando	28
Figura 5	–	Esquema geral dos componentes da <i>mixnet</i> e sua interação: o <i>frontend</i> , <i>backend</i> em Flask e os nós executando a API criptográfica	30
Figura 6	–	Tela inicial do frontend em CLI	30
Figura 7	–	Conteúdo da pasta local-scripts com os scripts de operação	31
Figura 8	–	Especificações da máquina utilizada para os processos de <i>benchmarking</i>	37
Figura 9	–	Gráfico do <i>benchmarking</i> de tempo de computação por número de votos na etapa de embaralhamento	38
Figura 10	–	Gráfico do <i>benchmarking</i> de tempo de computação por número de votos na etapa de decifração	38
Figura 11	–	Gráfico do <i>benchmarking</i> de tamanho dos arquivos de provas por número de votos na etapa de embaralhamento	39
Figura 12	–	Gráfico do <i>benchmarking</i> de tamanho dos arquivos de provas por número de votos na etapa de decifração	39
Figura 13	–	Gráfico do <i>benchmarking</i> de tempo de verificação das provas por número de votos na etapa de decifração	40
Figura 14	–	Gráfico do <i>benchmarking</i> de tempo de verificação das provas por número de votos na etapa de embaralhamento	40
Figura 15	–	Tela inicial, selecionando opção 1: Simular Votação	54
Figura 16	–	Tela de início da simulação de votação	54
Figura 17	–	Tela de início do setup da mix network Guardian	55
Figura 18	–	Execução dos scripts locais de setup nos nós 2 e 3 da mix network Guardian	56
Figura 19	–	Pasta do nó 1 da mix network Guardian após transferências de arquivos <i>protInfo</i> entre os nós	56
Figura 20	–	Execução dos scripts locais de merge nos nós 1, 2 e 3 da mix network Guardian	57
Figura 21	–	Tela de início da etapa de geração de chave pública	57
Figura 22	–	Execução dos scripts locais de geração de chave nos nós 2 e 3 da mix network Guardian	57
Figura 23	–	Chave pública resultante do processo de geração na mix network Guardian	58

Figura 24 – Tela de configuração dos votos válidos e duplicatas a serem gerados na eleição da demonstração	58
Figura 25 – Tela inicial, selecionando opção 2: Mixagem e Decifração	59
Figura 26 – Tela de início da configuração da mix network Shuffler	59
Figura 27 – Execução de scripts locais de setup da mix network Shuffler nos nós 2 e 3	60
Figura 28 – Pasta do nó 1 da mix network Shuffler após transferências de arquivos entre nós	60
Figura 29 – Execução de scripts locais de merge da mix network Shuffler nos nós 1, 2 e 3	61
Figura 30 – Execução de scripts locais de definição da chave pública da mix network Shuffler nos nós 1, 2 e 3	61
Figura 31 – Tela inicial da mix network Shuffler, selecionando opção 1: Executar Shuffle (misturar)	62
Figura 32 – Execução de scripts locais de mistura da mix network Shuffler nos nós 2 e 3	62
Figura 33 – Inicialmente o front-end apresenta mensagem de erro	63
Figura 34 – Após pressionar <i>Enter</i> na tela com erro, mensagem de sucesso aparece, finalizando etapa de mistura	63
Figura 35 – Pasta do nó 1 da mix network Shuffler após mistura (shuffle), copiando o arquivo gerado <i>shuffled</i>	64
Figura 36 – Pasta do nó 1 da mix network Guardian após transferência do arquivo <i>shuffled</i> da mix network Shuffler para os nós da mix network Guardian	65
Figura 37 – Tela de início da etapa de decifração, selecionando opção 2: Decifrar Votos	66
Figura 38 – Execução de scripts locais de decifração da mix network Guardian nos nós 2 e 3	66
Figura 39 – Tela de sucesso da etapa de decifração	67
Figura 40 – Execução de comandos de verificação das provas das etapas de decifração e de mistura	67
Figura 41 – Telas de login, perfil e inicial do aplicativo.	71
Figura 42 – Telas de aviso, FAQ e menu do aplicativo.	71
Figura 43 – Telas de depuração, mapa e QR Code do aplicativo.	72
Figura 44 – Fluxo de uso do aplicativo (Passos 1, 2 e 3).	72
Figura 45 – Fluxo de uso do aplicativo (Passo 4).	73

Sumário

1	INTRODUÇÃO	9
1.1	Motivação	9
1.2	Objetivos	9
1.3	Justificativa	10
1.4	Organização do Trabalho	10
2	ASPECTOS CONCEITUAIS	11
2.1	Mobilidade em Sistemas Eleitorais	11
2.2	Mix Networks	12
2.3	Criptografia Threshold	13
2.4	Provas de Conhecimento Zero (ZKPs)	14
2.5	Biblioteca Verificatum	14
2.6	Códigos QR	15
3	MÉTODO DO TRABALHO	17
3.1	Modelo Conceitual da Solução de Mobilidade	17
3.2	Estruturação do Projeto	19
3.3	Metodologia	20
3.4	Testes	22
4	ESPECIFICAÇÃO DE REQUISITOS	23
4.1	Requisitos Funcionais	23
4.2	Requisitos Não Funcionais	24
4.3	Restrições	25
4.4	Critérios de Sucesso	25
5	DESENVOLVIMENTO DO TRABALHO	27
5.1	Fluxo Implementado pela Solução	27
5.2	Arquitetura Completa da Solução	29
5.3	Módulo 1 — API em Java com Verificatum	31
5.4	Módulo 2 — <i>Frontend</i> e Simulação de Eleição	32
5.4.1	Funcionamento do <i>Frontend</i> em CLI	32
5.4.2	Simulação e Cifragem de Votos	33
5.5	Módulo 3 - Protótipo Móvel	33
5.5.1	Tecnologias e Ambiente de Desenvolvimento	34
5.5.2	Arquitetura Interna do Aplicativo	34

5.5.3	Fluxo do Eleitor no Aplicativo	35
5.5.4	Simulação dos Estados do Processo Eleitoral	35
5.5.5	Geração do Token de Mobilidade	36
5.5.6	Limitações Reais do QR Code e Motivação para a Divisão em Partes	36
5.5.7	Desafios Práticos na Renderização dos QR Codes	36
5.6	Testes e Avaliação	37
5.6.1	Resultados do benchmarking	37
6	CONSIDERAÇÕES FINAIS	43
6.1	Conclusões do Projeto de Formatura	43
6.1.1	Estado Final do Aplicativo	43
6.1.2	Estado Final da API	44
6.2	Contribuições	45
6.3	Perspectivas de Continuidade	46
	 REFERÊNCIAS	 47
	 APÊNDICES	 49
	APÊNDICE A – ENDPOINTS DA API EM JAVA	51
	APÊNDICE B – EXECUÇÃO COMPLETA DA DEMONSTRAÇÃO LOCAL (PASSO A PASSO)	53
	APÊNDICE C – CÓDIGO RESPONSÁVEL PELA CIFRAÇÃO	69
	APÊNDICE D – TELAS DO APLICATIVO PROTÓTIPO	71

1 Introdução

1.1 Motivação

As eleições são o evento central de um regime democrático. Nelas, a taxa de comparecimento do eleitorado é um fator central para a legitimidade não somente do governo eleito, mas do sistema democrático como um todo. Dessa forma, segue-se que um dos principais desafios para a manutenção de uma democracia saudável é a minimização da taxa de abstenção eleitoral.

No contexto das eleições brasileiras, especificamente, o voto é compulsório aos cidadãos adultos habilitados a exercê-lo. No entanto, o país ainda apresenta consistentemente uma taxa de abstenção significativa. Dentre as razões citadas por eleitores para não comparecerem ao pleito, destaca-se a dificuldade de acesso ao seu local designado de votação ([XAVIER, 2024](#)) - seja por ocasião de viagem, mudança, etc.

Para remediar situações como essas, atualmente, existem mecanismos no sistema eleitoral brasileiro como o voto em trânsito, ou a própria transferência do título de eleitor. Esses procedimentos devem ser feitos com alguns meses de antecedência à data da eleição. No caso do voto em trânsito, a solicitação deve ser feita obrigatoriamente presencialmente ([Tribunal Superior Eleitoral, 2022](#)), enquanto a transferência de título de eleitor pode ser feita online, porém mediante preenchimento de um formulário digital e apresentação de uma série de documentos ([Tribunal Superior Eleitoral, 2025b](#)).

Para além do atual sistema eleitoral, existem artigos que exploram novas soluções possíveis para a implementação de “mobilidade” nas eleições brasileiras, ou seja, permitir ao eleitor escolher com maior facilidade seu local de votação. Dentre esses, destacam-se “How to implement anywhere voting: A case study of brazil” ([KIMURA; ARAÚJO; SIMPLICIO, 2025](#)) e “Improving the Brazilian electronic voting system through the use of end-to-end verifiability” ([COMINETTI, 2024](#)) como os mais relevantes para o presente trabalho.

1.2 Objetivos

O objetivo deste trabalho é elaborar um sistema baseado no conceito criptográfico de *mix networks* para o tratamento e remoção segura, privada e auditável de eventuais votos duplos feitos em eleições com mobilidade. Conjuntamente, objetiva-se também o desenvolvimento de um protótipo de aplicativo de escolha de local de votação e geração de códigos *QR* para autenticação no local de votação escolhido.

1.3 Justificativa

Dado o contexto explorado na Seção 1.1, a realização deste trabalho constitui uma contribuição à elaboração de soluções para a mobilidade em eleições brasileiras, o que, por sua vez, visa a diminuição da abstenção eleitoral. Ao implementar sugestões de soluções propostas, principalmente, no artigo mencionado anteriormente, (KIMURA; ARAÚJO; SIMPLICIO, 2025), são fornecidos protótipos funcionais de aplicações, que servem como provas de conceito e base para futuros trabalhos de maior aprofundamento. Além disso, também são disponibilizados dados sobre parâmetros de desempenho, que podem embasar futuras análises de viabilidade de soluções análogas às aqui exploradas.

1.4 Organização do Trabalho

Este trabalho está organizado em seis capítulos, além das referências e apêndices.

O Capítulo 1 apresenta a introdução do tema, a motivação, os objetivos do projeto e a justificativa do estudo.

O Capítulo 2 reúne os aspectos conceituais necessários para o entendimento do trabalho, incluindo mobilidade em sistemas eleitorais, *mix networks*, criptografia de limiar, provas de conhecimento zero (ZKPs), a biblioteca Verificatum e o uso de QR Codes.

O Capítulo 3 descreve o método do trabalho, detalhando o planejamento das etapas, a metodologia adotada e os testes previstos.

O Capítulo 4 apresenta a especificação de requisitos da solução proposta, incluindo requisitos funcionais, requisitos não funcionais, restrições e critérios de sucesso.

O Capítulo 5 trata do desenvolvimento do trabalho, descrevendo o fluxo implementado pela solução, a arquitetura completa, a API em Java integrada à Verificatum, o *front-end* em linha de comando para simulação da eleição, o protótipo móvel e os testes e avaliações realizados.

O Capítulo 6 traz as considerações finais, as principais contribuições deste projeto de formatura e as perspectivas de continuidade.

Por fim, são apresentadas as referências bibliográficas utilizadas e os apêndices, que reúnem os *endpoints* da API em Java, o passo a passo da execução completa da demonstração local, o código responsável pela cifragem e as telas do aplicativo protótipo.

2 Aspectos Conceituais

Este capítulo apresenta os conceitos necessários para compreender a solução desenvolvida neste projeto. As seções seguintes descrevem, de forma objetiva, os mecanismos criptográficos, arquiteturais e operacionais que fundamentam o modelo de mobilidade na votação discutido no Capítulo 3 e implementado no Capítulo 5.

2.1 Mobilidade em Sistemas Eleitorais

A mobilidade em sistemas eleitorais refere-se à capacidade de os eleitores participarem do processo de votação de forma flexível, independentemente de sua localização geográfica ou de dispositivos específicos. Essa abordagem busca modernizar os sistemas tradicionais, permitindo que os cidadãos votem de maneira conveniente, seja por meio de dispositivos móveis, computadores pessoais ou terminais distribuídos em diferentes locais. A mobilidade é particularmente relevante em contextos onde a acessibilidade e a participação ampla são prioritárias, como em eleições com grande dispersão geográfica de eleitores ou em situações que exigem votação remota.

No entanto, a implementação de mobilidade em sistemas eleitorais apresenta desafios significativos, especialmente no que diz respeito à segurança, à privacidade e à integridade do voto. É fundamental garantir que os votos transmitidos por meio de dispositivos móveis ou redes não confiáveis sejam protegidos contra interceptações, fraudes e manipulações. Além disso, a identidade do eleitor deve ser verificada de forma confiável, sem comprometer seu anonimato. Esses requisitos tornam a mobilidade um tema complexo, que exige a adoção de protocolos robustos e auditáveis para assegurar a confiança no processo eleitoral. A mobilidade, quando implementada de forma adequada, tem o potencial de aumentar a participação eleitoral e modernizar os sistemas democráticos, desde que sejam atendidos os critérios de segurança e privacidade necessários.

Vale notar que no atual estado do sistema eleitoral brasileiro, o procedimento que mais se aproxima de responder às demandas que seriam atendidas por uma solução de mobilidade é o voto em trânsito. Este possibilita ao eleitor, com alguns meses de antecedência, comparecer a uma unidade de atendimento eleitoral portando um documento com foto e requerir a alteração do município em que irá votar para um diferente do qual se localiza seu domicílio eleitoral, somente durante a próxima eleição ([Tribunal Superior Eleitoral, 2022](#)).

2.2 Mix Networks

Mix networks são sistemas de comunicação projetados para garantir o anonimato e a privacidade dos usuários ao transmitir mensagens em redes digitais. (COMINETTI, 2024) Essas redes operam por meio de uma série de nós intermediários, que processam e retransmitem as mensagens de forma a obscurecer a relação entre o remetente e o destinatário.

O funcionamento básico de uma *mix network* típica envolve a criptografia das mensagens em camadas, de modo que cada nó remove uma camada de criptografia antes de retransmitir a mensagem ao próximo nó. Além disso, os nós reordenam as mensagens recebidas, dificultando a correlação entre a entrada e a saída de dados em cada nó. Esse processo de reordenação é essencial para impedir que um adversário, mesmo com capacidade de monitorar parte da rede, consiga rastrear o fluxo de comunicação ou identificar os participantes. A imagem 1 mostra um esquema do funcionamento de uma *mix network* típica.

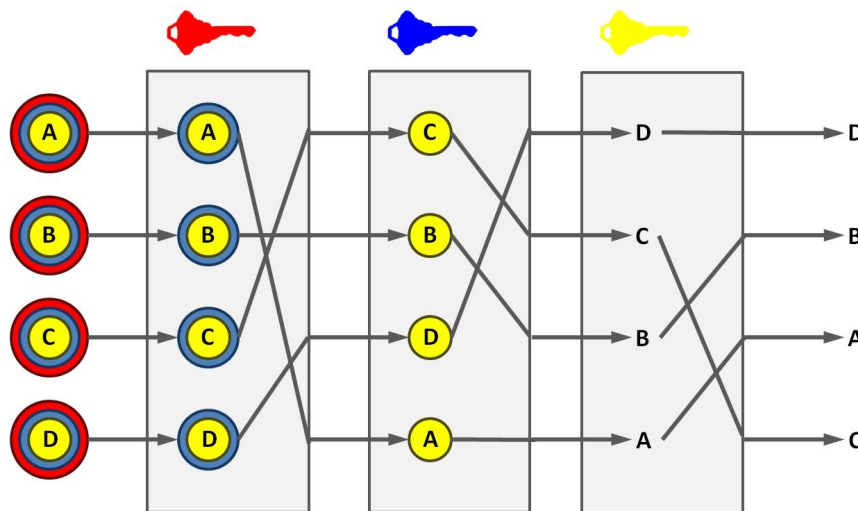


Figura 1 – Esquema simplificado de uma *mix network*

Fonte: <https://www.makeuseof.com/what-is-a-mixnet/>

Já as *mix networks* de recifração funcionam de forma um pouco diferente. Nelas, só é utilizada a chave pública do recipiente, de forma que cada nó da *mix network* não possa decifrar as mensagens, porém seja capaz de recifrá-las de modo que as saídas não possam ser vinculadas às respectivas entradas. Ao mesmo tempo, a chave de decifração (chave privada do recipiente) se mantém inalterada. A Figura 2 mostra um esquema do funcionamento básico de uma *mix network* de recifração.

A aplicação de *mix networks* é particularmente relevante em cenários onde a privacidade e a integridade das comunicações são críticas, como em sistemas de votação

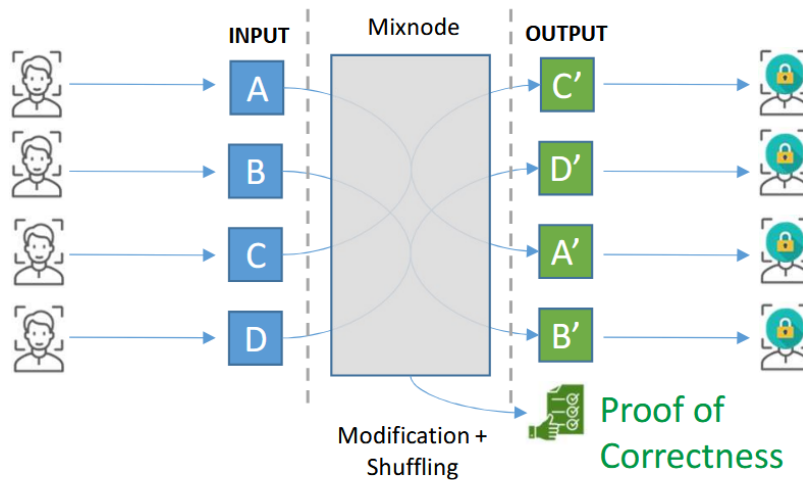


Figura 2 – Esquema simplificado de uma *mix network* de recifração

Fonte: (COMINETTI, 2024)

eletrônica. Nesse contexto, a necessidade de garantir o anonimato dos eleitores, ao mesmo tempo em que se assegura a veracidade e a transparência dos votos, torna as *mix networks* uma solução tecnicamente viável. Ao utilizar técnicas de criptografia, essas redes podem proteger a identidade dos eleitores durante a transmissão dos votos, evitando a associação de um voto a um indivíduo específico.

2.3 Criptografia Threshold

A criptografia limiar distribui entre vários participantes a capacidade de realizar operações criptográficas sensíveis, impedindo que qualquer servidor isolado possua a chave completa ou consiga agir sozinho. Como descrito por Desmedt (DESMEDT, 1994), esse paradigma difere dos esquemas tradicionais de compartilhamento de segredo porque a chave nunca é reconstruída; em vez disso, cada participante fornece uma contribuição parcial, e apenas a combinação de contribuições autorizadas produz o resultado válido.

Essa abordagem aumenta a resiliência do sistema ao reduzir riscos de comprometimento, evitar pontos únicos de falha e permitir operação segura mesmo quando alguns servidores falham ou agem de forma indevida. Para este trabalho, essa propriedade é central, pois sustenta a lógica de cooperação entre guardiões na *mix network*, garantindo segurança estrutural sem depender de confiança absoluta em qualquer nó individual.

2.4 Provas de Conhecimento Zero (ZKPs)

As Zero-Knowledge Proofs (ZKPs) são protocolos criptográficos que permitem comprovar a veracidade de uma afirmação sem revelar qualquer informação adicional sobre o segredo envolvido. De acordo com Hasan ([HASAN, 2019](#)), um ZKP satisfaz três propriedades fundamentais: completude, garantindo que um provador honesto sempre convença um verificador honesto; solidez, assegurando que afirmações falsas não sejam aceitas; e *zero-knowledge*, indicando que nada além da veracidade da afirmação é revelado durante o processo.

A principal utilidade das ZKPs, no contexto deste trabalho, está na verificabilidade: é possível provar que certas operações criptográficas ocorreram corretamente — por exemplo, um embaralhamento de votos ou uma decifração parcial — sem expor dados sensíveis, como a correspondência entre eleitor e voto. Sistemas de mixnets modernos, como a biblioteca *Verificatum*, utilizam exatamente esse princípio para fornecer provas públicas de correção, garantindo que cada etapa do processamento possa ser auditada independentemente, mas sem comprometer a privacidade dos elementos manipulados.

Assim, ZKPs oferecem a base formal para que o sistema preserve duas propriedades simultâneas: (1) transparência, pois todas as operações críticas são acompanhadas de provas verificáveis, e (2) sigilo, pois nenhuma informação adicional é revelada. Esse equilíbrio é essencial para permitir verificabilidade fim-a-fim em processos eleitorais sem violar o anonimato do voto.

2.5 Biblioteca Verificatum

Para a implementação do componente de mixagem do sistema proposto, foi adotada a biblioteca *Verificatum*, uma solução consolidada de código aberto para execução de *mixnets* baseadas em re-criptação com provas de conhecimento zero ([VERIFICATUM, 2024](#)).

A biblioteca oferece uma implementação robusta do criptossistema ElGamal sobre curvas elípticas e suporta tanto provas interativas quanto não interativas. Sua arquitetura modular é dividida em seis componentes principais:

- **GMPMEE** e **VEC**: extensões para exponenciação eficiente e operações em curvas elípticas.
- **VMGJ** e **VE CJ**: interfaces Java para as bibliotecas anteriores.
- **VCR**: núcleo das rotinas de criptografia e mixagem.
- **VMN**: implementação dos protocolos de mixnet propriamente ditos.

No contexto do sistema de votação com mobilidade, foram utilizadas três funcionalidades centrais da biblioteca:

1. **Re-encryption Shuffle**: reembaralhamento de votos cifrados via recifração com o mesmo par de chaves, eliminando qualquer vínculo observável entre entrada e saída.
2. **Proof of Correct Shuffle**: geração de provas de conhecimento zero (do tipo Wikström-Terelius) que asseguram a correção do embaralhamento sem revelar a permutação utilizada.
3. **Proof of Correct Decryption**: geração de provas de decifração correta, garantindo que cada voto foi corretamente decifrado sem alteração do conteúdo original.

Essas funcionalidades permitem a implementação de uma *mix network* com verificação pública não apenas da integridade dos votos, mas também da preservação de sua confidencialidade. A verificação das provas pode ser realizada de forma assíncrona e auditável por qualquer observador externo, respeitando os princípios de transparência e auditabilidade eleitoral. Além disso, a Verificatum fornece suporte a pré-computações e testes automatizados para validação de cada componente, sendo compatível com bibliotecas de alta performance como GMP e OpenSSL. O projeto está disponível publicamente em seu repositório oficial e no site institucional: <<https://www.verificatum.org>>.

Em suma, a adoção da Verificatum oferece uma base confiável, auditável e eficiente para a execução de *mix networks* de recifração em contextos restritos como o das eleições brasileiras, contribuindo diretamente para a segurança e verificabilidade do sistema proposto (COMINETTI, 2024).

2.6 Códigos QR

Tecnicamente, um QR Code é formado por uma matriz de módulos pretos e brancos organizada em versões numeradas de 1 a 40. Cada versão corresponde a um tamanho fixo do símbolo, aumentando em quatro módulos por lado a cada incremento. A versão mais simples, Version 1, possui 21×21 módulos, enquanto a versão máxima, Version 40, chega a 177×177 módulos. Assim, versões maiores oferecem maior capacidade de armazenamento, ainda que exijam área física superior para impressão e leitura (DENSO WAVE Incorporated, 2025).

A escolha da versão adequada depende diretamente do volume de dados a ser codificado e do nível de correção de erros desejado. O QR Code utiliza códigos de correção Reed-Solomon, disponíveis em quatro níveis — L, M, Q e H. Níveis mais altos aumentam a resiliência a danos físicos, porém reduzem a quantidade útil de informação armazenável (TIWARI, 2016).

De modo geral, o QR Code oferece grande flexibilidade ao permitir ajustar simultaneamente três fatores fundamentais: (i) o volume de dados codificado, (ii) o nível de tolerância a erros e (iii) o tamanho físico do símbolo. Essa combinação torna o QR Code adequado tanto para pequenos textos quanto para cargas mais complexas, como tokens, chaves ou fragmentos de informações criptográficas — aspectos diretamente relacionados às aplicações discutidas nesta monografia.

3 Método do trabalho

Este capítulo tem como objetivo relatar o processo de desenvolvimento do trabalho, dispondo trabalhos anteriores que constituíram inspirações e as fases elaboradas para sua construção.

3.1 Modelo Conceitual da Solução de Mobilidade

O projeto foi concebido a partir de discussões previamente realizadas sobre mobilidade no cenário das eleições brasileiras, especialmente o trabalho já mencionado anteriormente "How to Implement Anywhere Voting: A Case Study of Brazil" (KIMURA; ARAÚJO; SIMPLICIO, 2025). Nesse artigo, os autores exploram soluções para viabilizar o voto em qualquer local, mesmo em contextos com infraestrutura limitada, como é o caso de muitas regiões brasileiras. Dentre as alternativas analisadas, destaca-se a proposta de remoção de eventuais votos duplos durante a apuração, utilizando técnicas como *mix networks* e criptografia de limiar para preservar a privacidade do eleitor e garantir a integridade do processo. Essa abordagem, que busca detectar e eliminar votos repetidos sem exigir conectividade contínua com a internet, serviu como principal inspiração para o desenvolvimento do presente trabalho.

O artigo apresentou um panorama da solução de mobilidade que envolve remoção de votos duplos, e está disposto abaixo na Figura 3. A seguir, ele será detalhado, visto que o projeto se originou a partir dele. Vale notar que o artigo entra nos pormenores do contexto, das escolhas feitas que envolveram tal solução e suas limitações, de modo que aqui se relata apenas no que ela constitui e, principalmente, as decisões que foram tomadas para transformá-la num projeto de engenharia de *software*.

Conforme descrito no artigo, “A solução pode ser estruturada em três fases principais: a geração de *tokens*, o processo de votação e a apuração dos resultados” (tradução livre), e é essa divisão que serviu como ponto de partida para a concepção deste trabalho. A primeira fase, a de geração de *tokens*, envolve a participação de um “aplicativo móvel eleitoral”, criado especificamente para eleitores que desejem participar do mecanismo de voto móvel. O aplicativo fornece a usuários de uma eleição um *tokenId* e dados biométricos cifrados, em formato de código *QR* assinado digitalmente.

Posteriormente, na “fase de votação”, o eleitor usaria esse mesmo código *QR* como ferramenta para tornar possível o voto móvel: seus dados biométricos seriam usados para autenticação, enquanto o *tokenId* seria justamente aquilo usado para controle dos votos duplos na fase de apuração.

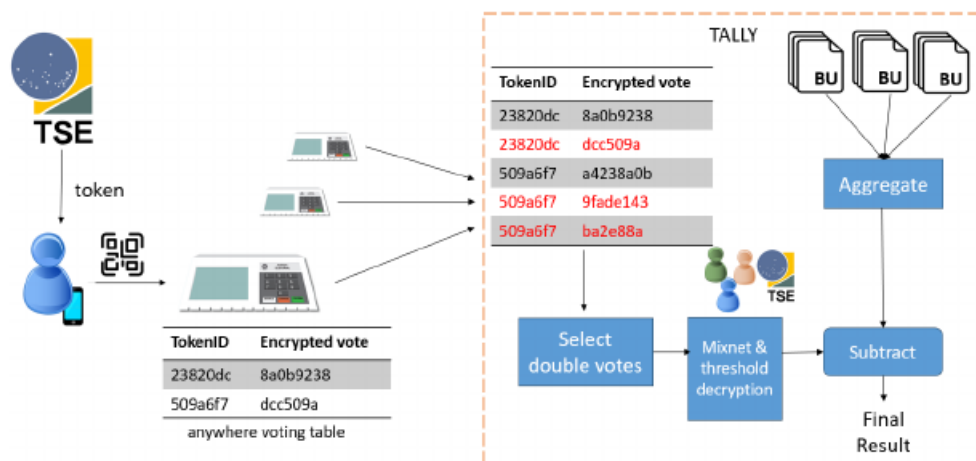


Figura 3 – Panorama da solução de mobilidade estudada baseada na remoção de votos duplos

Fonte: (KIMURA; ARAÚJO; SIMPLICIO, 2025)

Esse controle de remoção de votos duplos se daria da maneira disposta a seguir. Ao realizar um voto, a urna então guarda a relação entre *tokenID* e o voto cifrado. Ao final da eleição, todos esses votos são agregados e processados para serem considerados na fase de apuração. Isso resultaria numa tabela com todos *tokenID* relacionados a seus respectivos votos cifrados. Todos os votos duplos seriam detectados utilizando o próprio *tokenID* e, em seguida, extraídos utilizando algum critério previamente definido (por exemplo, priorizar votos que foram registrados primeiro, e extrair os posteriores).

Com votos duplos extraídos, constitui-se uma nova tabela composta somente pelos votos cifrados daqueles que foram considerados votos duplos (desvinculados dos *tokenIDs*). São esses votos provenientes de duplicados removidos que alimentariam a *mix network*. Essa *mix network* é composta por algumas entidades que estariam participando da cerimônia de contabilização dos votos. Os votos são então misturados por essa *mix network* e, em seguida, decifrados por criptografia de limiar. Finalmente, tais votos decifrados seriam subtraídos do resultado preliminar (que contabiliza todos os votos, inclusive duplos), tendo-se, então, o resultado final da eleição.

Em suma, essa solução é baseada na identificação de votos duplos durante a etapa de apuração. Cada eleitor precisa de um *token* para fazer uso do voto móvel, do qual a urna realiza a leitura. Esse *token* é guardado juntamente com o voto cifrado. Ao final da eleição, todos esses votos são agregados e os votos duplos são identificados pelo *tokenID* repetido. Tais votos são extraídos e misturados utilizando o protocolo de *mix network*, o qual garante a quebra da associação entre *tokenID* e voto cifrado. Posteriormente são decifrados utilizando criptografia de limiar e removidos do resultado preliminar.

3.2 Estruturação do Projeto

O panorama apresentado anteriormente, proveniente do artigo mencionado, serviu como base para estruturar as fases deste trabalho. Inicialmente, identificou-se duas partes as quais poderiam ser desenvolvidas concomitantemente. Tratam-se do desenvolvimento de um protótipo de aplicativo móvel e a implementação da *mix network*.

O aplicativo móvel, como dito, é aquele responsável pelo fornecimento de *tokens* a eleitores que fizessem uso do voto móvel, através do formato de um código *QR*. Já a *mix network* é fundamental para garantir a privacidade e integridade no processo eleitoral (COMINETTI, 2024) no momento de remover os votos duplos. Esse protocolo embaralha criptograficamente os votos associados a *tokenIDs* repetidos e produz provas verificáveis de que nenhum voto foi modificado, removido ou inserido. Após esse embaralhamento, os votos são decifrados por meio de um esquema de criptografia de limiar, permitindo a remoção segura dos votos duplos sem revelar qualquer vínculo entre voto e eleitor. Dessa forma, a *mix network* assegura que o processo de eliminação de votos duplos preserva o sigilo individual e permanece auditável de ponta a ponta.

A tese de doutorado “Improving the Brazilian electronic voting system through the use of end-to-end verifiability” (COMINETTI, 2024) explora em detalhes como aplicar verificabilidade fim-a-fim no contexto de eleições brasileiras ¹. O trabalho envolve, dentre outros aspectos, um *pipeline* criptográfico compatível com as restrições das urnas eletrônicas brasileiras, envolvendo técnicas como *mix networks* e provas de embaralhamento, que podem ser integradas de forma verificável ao processo eleitoral. Para implementar a *mix network* foi utilizada a biblioteca *Verificatum*, que é a mesma que se pretende utilizar neste trabalho.

Conforme descrito, a biblioteca “foi utilizada em eleições online reais em Israel (Wombat), Espanha (Agora Voting), Noruega (Scytl) e Estônia”, sendo, portanto, uma referência consolidada para implementação de protocolos de embaralhamento criptográfico em contextos eleitorais. Além de fornecer a rotina central de *mix network*, a *Verificatum* implementa toda a infraestrutura necessária para um embaralhamento verificável: re-fracção eficiente, provas de mistura (*shuffle*), módulos de pré-computação para ganho de desempenho e suporte completo a decifração distribuída com verificação pública. Trata-se, portanto, de uma solução madura, auditável e empregada em cenários reais de votação, o que elimina a necessidade — arriscada e tecnicamente custosa — de desenvolver rotinas criptográficas críticas do zero. Por essas razões, este trabalho faz uso da biblioteca *Verificatum* (VERIFICATUM, 2024) como base para a implementação da *mixnet*.

¹ Verificabilidade fim-a-fim (E2E) numa eleição diz respeito a uma propriedade de sistemas de votação garantida por protocolos criptográficos que permitem ao eleitor verificar que seu voto foi registrado e contado corretamente, sem comprometer o sigilo (U.S. Election Assistance Commission, 2023).

3.3 Metodologia

A metodologia adotada no projeto seguiu um modelo incremental, priorizando inicialmente contato com conceitos simples e evoluindo gradualmente para uma *pipeline* completa. Dessa maneira, em um primeiro momento, foi necessário compreender os conceitos básicos envolvidos na solução apresentada anteriormente. Sobretudo, dos mecanismos que envolvem uma *mix network*, suas etapas, limitações e funcionalidades. Inclusive, considerando que o *Verificatum* faz uso do sistema *ElGamal*, foi necessário também compreender brevemente do que se trata.

Para as funcionalidades básicas da *mix network* do *Verificatum* (VMN), seu manual oficial “User Manual for the Verificatum Mix-Net“ (AB, 2022) foi referência para compreender sua capacidade e todas as ferramentas que ele articula. Trata-se de uma *mix network* de recifração baseada no sistema ElGamal, escrita usando a linguagem de programação Java com trechos em C para maior eficiência. Nota-se que o sistema VMN implementa, além do shuffle de recifração, fases de geração distribuída de chaves, decifração com limiar, pré-computação para aceleração, e a produção automática de provas de conhecimento zero universalmente verificáveis.

Além do estudo teórico, realizou-se também atividades práticas com a biblioteca. Isso envolveu testes com a *demo* oficial². Trata-se de uma demonstração que fornece um mini-ambiente completo que simula uma eleição fictícia processada por uma *mix network*. Ela cria três servidores de mistura (*mix servers*) independentes, gera o conjunto de parâmetros compartilhados e executa a geração distribuída da chave pública. Em seguida, produz *ciphertexts* de exemplo que representam votos cifrados e aciona o protocolo de mistura, no qual cada servidor realiza o *shuffle* com recifração e a decifração conjunta. Por fim, a *demo* gera as provas não-interativas de conhecimento zero correspondentes e valida toda a execução com o verificador oficial (VMN). Assim, ela funciona como um ambiente mínimo que reproduz o comportamento real de uma *mix network* do *Verificatum*, servindo como referência prática para a estruturação e operação do sistema. Também se experimentou escrevendo algumas rotinas para testar o grau de complexidade envolvido em eventuais adaptações ou extensões de código.

Além da documentação oficial do *Verificatum*, também foi analisado o repositório público do projeto Trustfull, mantido pelo KTH Theoretical Computer Science Department, que disponibiliza um demonstrador completo de votação eletrônica baseado na *mixnet* *Verificatum*³. Nesse sistema, a camada criptográfica é executada em múltiplos servidores na nuvem, orquestrados por scripts em Python que realizam o *deploy*, inicialização da eleição, coleta dos *ciphertexts* e apuração distribuída, enquanto um *front-end* web em Flask funciona

² Detalhes da *demo* podem ser lidos em <<https://github.com/verificatum/verificatum-vmn/tree/master/demo/mixnet>>

³ Repositório disponível em <<https://github.com/kth-tcs/trustfull-demonstrator>>.

como servidor de coleta de votos, utilizando a Verificatum JavaScript Cryptography Library⁴ para cifrar os votos diretamente no navegador antes de enviá-los ao servidor. Embora o contexto do projeto seja distinto do deste trabalho, sua arquitetura ajudou a visualizar como organizar, de forma modular, a interação entre a camada criptográfica, o *back-end* de coleta e a interface de votação, além de servir como referência concreta de como implementar a cifragem de votos no cliente e expor uma API de orquestração para todo o fluxo da mixnet.

Posteriormente, considerou-se alternativas para traduzir a implementação do Verificatum no projeto de *software* almejado. Isso envolveu testar diferentes interfaces de interação de usuário para com a *mix network*. Por exemplo, dispor as informações da *mix network* numa plataforma web, uma interface gráfica ou uma aplicação em linha de comando. Ademais, foram experimentadas diferentes filosofias de projeto, variando, por exemplo, entre um sistema mais didático e visual ou entre uma solução mais fiel simulando um ambiente realista de uma aplicação realmente usada por entidade eleitoral.

Finalmente, optou-se por integrar as funcionalidades do Verificatum a uma API própria, a qual alimentaria um *front-end* em interface de linha de comando. Para tanto, cada operação essencial do Verificatum - *setup*, mistura e decifração de limiar - foram integradas a essa API modularmente. Isso garante uma separação entre a camada criptográfica, a camada de controle do *front-end* e a própria interface de usuário.

Esse processo incluiu a construção de protótipos intermediários, testes exploratórios em ambiente local e revisões com o co-orientador para garantir que o sistema permanecesse coerente com os objetivos do trabalho. A cada iteração, buscou-se refinar o entendimento do protocolo, ajustar o escopo às limitações observadas e validar a viabilidade das escolhas técnicas.

Complementarmente ao desenvolvimento e análise da *mix network*, a implementação do protótipo móvel seguiu uma metodologia iterativa inspirada nos princípios do *Design Sprint* (KNAPP; ZERATSKY; KOWITZ, 2016). O processo foi organizado em ciclos curtos de concepção, prototipação e validação interna, permitindo testar rapidamente diferentes fluxos de navegação, formatos de apresentação do *token* e mecanismos de comunicação entre telas. Cada iteração buscou refinar exclusivamente a experiência do eleitor no contexto do voto móvel, priorizando clareza, simplicidade operacional e fidelidade ao fluxo definido na arquitetura do sistema. Essa abordagem incremental possibilitou incorporar, de forma contínua, os aprendizados obtidos nas etapas anteriores — tanto os provenientes do estudo da *mix network* quanto das restrições práticas do uso em dispositivos móveis — assegurando que o aplicativo evoluísse em alinhamento com os requisitos funcionais e o cenário proposto.

⁴ A Verificatum JavaScript Cryptographic Library (VJSC) é a biblioteca oficial em JavaScript do projeto Verificatum, utilizada para executar, no lado do cliente, operações criptográficas compatíveis com o sistema de votação.

3.4 Testes

Os testes realizados ao longo do projeto tiveram como objetivo verificar o funcionamento correto dos módulos implementados e validar a coerência do fluxo criptográfico, visando, em última instância, constatar a viabilidade das soluções desenvolvidas. Inicialmente, foram conduzidos testes exploratórios para confirmar a execução das etapas fundamentais do *Verificatum*, como geração de chaves, embaralhamento e decifração de limiar. Em seguida, testou-se a integração entre os componentes desenvolvidos — API criptográfica (*mix network*), *front-end* em interface de linha de comando, bem como seu *back-end* — assegurando que a troca de arquivos e execução de comandos ocorresse de maneira estável.

Além disso, foram executadas simulações com diferentes números de votos, para avaliar o comportamento da *pipeline* como um todo e identificar possíveis falhas na orquestração. Esses testes permitiram ajustar detalhes do fluxo e garantir que o protótipo cumprisse seu papel demonstrativo com confiabilidade.

Finalmente, foram estabelecidos também os critérios finais de viabilidade para ambas as soluções, considerando o cenário da implementação de mobilidade nas eleições brasileiras. Considerou-se que, atualmente, os resultados das eleições no Brasil começam a ser divulgados no próprio dia do pleito, logo após o encerramento da votação ([Tribunal Superior Eleitoral, 2025a](#)). Com base nesse contexto, adotou-se como referência um tempo de processamento na ordem de até 3 horas para o fluxo completo da *mix network*, de forma a não comprometer a divulgação dos resultados no mesmo dia ou, no máximo, até o dia seguinte. Quanto ao aplicativo, estabeleceu-se como critério a capacidade de inserção de todas as informações necessárias para o voto móvel de um eleitor dentro de códigos *QR* e posterior leitura bem sucedida destes, constituindo uma prova de conceito da solução.

4 Especificação de Requisitos

Este capítulo apresenta a especificação de requisitos elaborada para orientar o desenvolvimento do projeto. Eles estão organizados em duas categorias principais: requisitos funcionais e requisitos não funcionais. Além disso, são listadas as restrições adotadas e os critérios de sucesso utilizados para validar o resultado final.

4.1 Requisitos Funcionais

- **RF1 - Execução de Protocolos via API**

A API deverá permitir a execução completa dos principais protocolos criptográficos da biblioteca Verificatum, incluindo:

- Geração de arquivos de configuração da *mix network*;
- Execução de protocolos como geração de chave pública, embaralhamento, decifração e verificação de provas de conhecimento zero (ZKPs);
- Implementação de comunicação via *bulletin board* entre os servidores.

- **RF2 - Monitoramento e Feedback em Tempo Real**

A API deverá capturar e disponibilizar logs e saídas de execução, permitindo que a interface CLI acompanhe o progresso de cada etapa dos protocolos.

- **RF3 - Integração com Interface CLI**

A interface CLI deverá interagir com a API para iniciar e monitorar execuções da mix-net. Deve ser possível executar todas as etapas da eleição experimental sem recorrer a comandos do Verificatum diretamente no terminal.

- **RF4 - Modularidade e Execução Assíncrona**

A arquitetura deve permitir que cada servidor da *mix network* execute sua etapa de forma independente, sem necessidade de sincronização rígida. A troca de resultados poderá ocorrer via arquivos protegidos.

- **RF5 - Processamento de Votos Duplicados**

A *mix network* deverá receber como entrada apenas votos identificados como duplos. Ela deverá embaralhar e decifrá-los preservando a privacidade, de forma a possibilitar sua contabilização individual e posterior subtração dos totais da eleição.

- **RF6 - Geração de Token Eleitoral no Aplicativo**

O aplicativo móvel deverá gerar um token criptográfico único (simulado) representando o eleitor, contendo identificador, metadados e assinatura simulada ou placeholder.

- **RF7 - Exibição do Token via QR Code**

O aplicativo deverá exibir o token em formato de QR Code, permitindo que o módulo de leitura (CLI ou urna simulada) capture seu conteúdo. Caso necessário, deve suportar divisão em múltiplas partes.

- **RF8 - Fluxo Simulado de Voto Móvel**

O aplicativo deverá apresentar um fluxo didático representando o processo de voto móvel, incluindo telas de identificação simulada, informações do eleitor, explicação do processo e emissão do token.

- **RF9 - Compatibilidade com o Backend**

O formato do token emitido pelo aplicativo deverá ser compatível com a API e com o back-end Flask, permitindo ingestão direta para testes de duplicatas e integração com o fluxo da mix-net.

- **RF10 - Funcionamento Offline do Aplicativo**

O aplicativo deverá funcionar completamente de forma offline, refletindo o cenário real de urnas eletrônicas sem conexão entre si.

4.2 Requisitos Não Funcionais

- **RNF1 - Compatibilidade com Linux (Ubuntu)**

Todo o desenvolvimento será realizado para sistemas baseados em Linux, com testes executados em Ubuntu 22.04 ou superior.

- **RNF2 - Segurança de Arquivos e Dados**

Os arquivos gerados (chaves, logs, votos) deverão ser armazenados com permissões restritas. O sistema não poderá permitir a quebra da privacidade dos votos.

- **RNF3 - Usabilidade da Interface CLI**

A CLI deverá ser simples, intuitiva e adequada ao caráter demonstrativo do projeto, facilitando a visualização das etapas da mix-net.

- **RNF4 - Suporte à Medição de Desempenho**

A plataforma deverá permitir a medição do tempo de execução de cada fase da mix-net, possibilitando análise comparativa e coleta de métricas.

- **RNF5 - Compatibilidade com Android**

O aplicativo móvel deverá ser compatível com dispositivos Android, desenvolvido em React Native com Expo.

- **RNF6 - Simplicidade e Clareza Visual**

A interface do aplicativo deverá ser leve, simples e acessível ao usuário, refletindo o caráter didático da demonstração.

- **RNF7 - Segurança do Token Simulado**

O token exibido pelo aplicativo não deverá conter dados sensíveis reais e deverá seguir um formato previsível e seguro para demonstrações.

4.3 Restrições

- O código-fonte da biblioteca Verificatum não será modificado. A interação ocorrerá via linha de comando ou chamadas internas Java.
- Os testes iniciais simularão múltiplos mix-servers em um único computador. Execuções distribuídas em rede local serão feitas apenas em etapas posteriores.
- A primeira versão funcional contemplará apenas três mix-servers e um número limitado de votos para validação.

4.4 Critérios de Sucesso

- A API executa de forma autônoma todos os comandos de configuração e execução da mix-net.
- A CLI permite iniciar, acompanhar e visualizar cada fase do processo com clareza.
- O sistema identifica, embaralha e revela votos duplicados com privacidade e de maneira confiável.
- O aplicativo móvel gera corretamente tokens e QR Codes compatíveis com o fluxo criptográfico.
- Relatórios de desempenho podem ser gerados com métricas comparativas de tempo, carga e escalabilidade, validando a viabilidade da solução.

5 Desenvolvimento do Trabalho

Este capítulo descreve a implementação prática da solução proposta, detalhando a arquitetura dos componentes, as tecnologias utilizadas e os mecanismos que permitem a execução completa do fluxo criptográfico.

A solução foi implementada a partir de três módulos principais:

1. API criptográfica: responsável por disponibilizar ao *frontend* todas as funcionalidades necessárias para a execução da *mixnet*.
2. *Frontend*: responsável por orientar o usuário às etapas da execução da *mixnet*, envolvendo também uma simulação de uma eleição utilizando a mesma chave pública da *mixnet*;
3. Protótipo Móvel: responsável pela orientação do usuário sobre o conceito de “Voto Móvel”, geração de *tokens* e sua exibição em formato de código *QR*;

As seções seguintes apresentam o funcionamento de cada módulo, tecnologias utilizadas e a forma como interagem para compor o processo de mobilidade e privacidade dos votos.

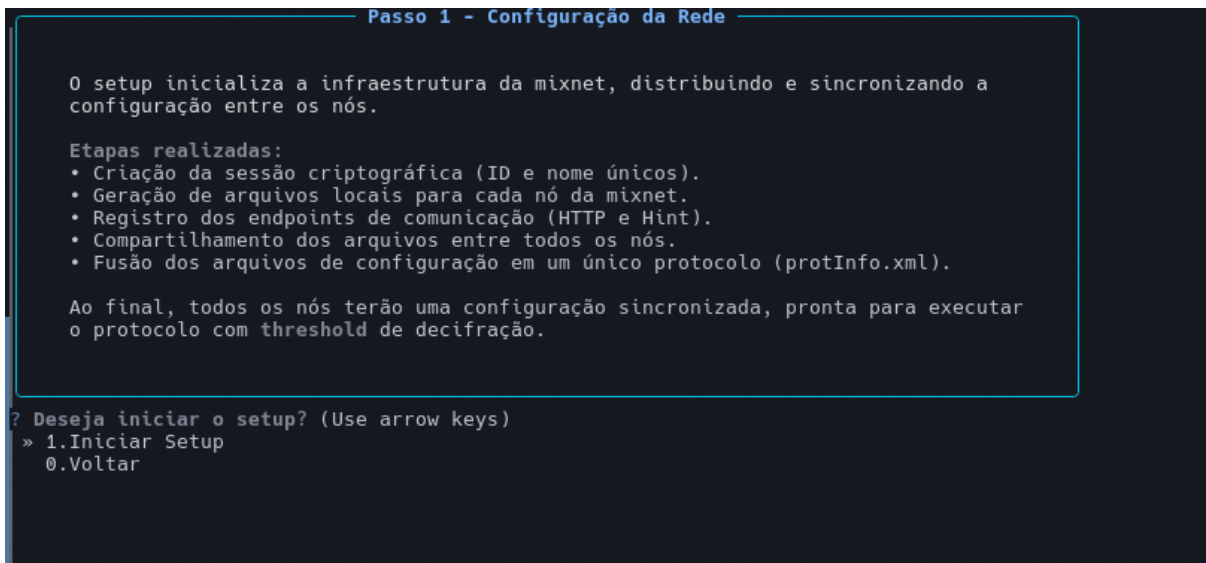
5.1 Fluxo Implementado pela Solução

Primeiramente, dispõe-se a seguir o fluxo com todas as etapas executadas e devidamente apresentadas no *frontend*:

1. Inicialização do *setup* dos nós guardiões, que gerarão os parâmetros necessários para a criação da chave pública;
2. Execução da geração da chave pública distribuída;
3. Cifragem dos votos utilizando a chave pública;
4. Envio dos votos cifrados a um servidor Flask, que registra o arquivo e realiza o processamento para identificação de duplicatas;
5. Execução do *setup* dos nós embaralhadores (*shufflers*);
6. Envio da tabela de votos duplos processados ao primeiro nó embaralhador;
7. Execução do processo de embaralhamento (*shuffle*) pelos nós, em sequência;

8. Visualização, pelo *frontend*, dos votos antes e depois do embaralhamento;
9. Execução da decifração distribuída e apresentação do conjunto final de votos;

A ideia foi, então, permitir que o usuário conseguisse interagir com cada etapa, de maneira que as telas apresentassem claramente o que cada etapa está executando. Por exemplo, na figura 4 a seguir, está disposto um exemplo de uma dessas telas. No caso trata-se da tela para realizar o setup inicial.



```
Passo 1 - Configuração da Rede

O setup inicializa a infraestrutura da mixnet, distribuindo e sincronizando a
configuração entre os nós.

Etapas realizadas:
• Criação da sessão criptográfica (ID e nome únicos).
• Geração de arquivos locais para cada nó da mixnet.
• Registro dos endpoints de comunicação (HTTP e Hint).
• Compartilhamento dos arquivos entre todos os nós.
• Fusão dos arquivos de configuração em um único protocolo (protInfo.xml).

Ao final, todos os nós terão uma configuração sincronizada, pronta para executar
o protocolo com threshold de decifração.

? Deseja iniciar o setup? (Use arrow keys)
> 1.Iniciar Setup
  0.Voltar
```

Figura 4 – Tela do setup inicial do frontend em linha de comando

Fonte: Elaboração Própria

Além da sequência operacional, o fluxo mostra a integração entre a API criptográfica, o *frontend* e o módulo de simulação de votos. Juntos, esses componentes reproduzem o cenário descrito no Capítulo 3, no qual votos duplicados são detectados, processados de forma anônima e removidos antes da totalização.

As etapas iniciais (1 e 2) formam a preparação criptográfica. Nessa fase, os nós guardiões executam o protocolo de geração distribuída de chaves, produzindo a chave pública que será usada ao longo do fluxo. Como o segredo é dividido entre os nós, nenhum deles consegue decifrar votos individualmente.

Na etapa de cifragem (3), um módulo denominado de *MockElection* gera votos sintéticos e os cifra com a chave pública. A simulação é independente do restante da pipeline, o que permite testar diferentes escalas de eleição sem alterar o processamento posterior.

O envio ao servidor Flask (4) inicia o processamento de votos duplos. O servidor identifica votos duplicados com base no *tokenID* e encaminha apenas os *ciphertexts*

relevantes para a *mixnet*. Ele atua como ponto de transição entre a geração dos votos e o início do processamento criptográfico.

A segunda parte do fluxo (5 a 9) corresponde ao processamento da *mixnet*. Após o *setup* dos nós embaralhadores, os votos duplicados passam por recifração sequencial, rompendo qualquer vínculo observável entre o *tokenID* e o voto cifrado. O *frontend* exibe os conjuntos antes e depois do embaralhamento, facilitando a verificação das alterações produzidas por cada nó.

Na etapa de decifração distribuída, os nós guardiões revelam apenas o conteúdo dos votos duplicados, sem recuperar associações individuais. Esses votos são então removidos da contagem preliminar, resultando no conjunto final de votos válidos.

O fluxo completo evidencia como os módulos da solução se combinam para produzir um processo íntegro e verificável. Ele também permite que cada etapa seja observada e executada isoladamente, o que facilita análise, depuração e reprodução do experimento.

5.2 Arquitetura Completa da Solução

A Figura 5 apresenta a arquitetura geral utilizada para executar o fluxo descrito na Seção 5.1. O diagrama destaca os três nós contendo a API criptográfica que executa Verificatum, o servidor Flask e a interface em linha de comando, bem como o papel de cada componente na execução coordenada da *mixnet*.

A solução utiliza três nós independentes, cada um executando uma instância da API criptográfica do Verificatum. Esses nós são responsáveis pelas etapas locais de *setup*, *shuffle* e decifração parcial. A separação física ou lógica entre os nós permite reproduzir o modelo distribuído exigido pelo protocolo, no qual nenhuma entidade isolada possui informação suficiente para decifrar votos.

Em uma quarta máquina é executado o servidor Flask, que atua como camada de orquestração. Ele recebe arquivos produzidos ao longo do fluxo, aciona as operações necessárias na API Java e fornece ao *frontend* o estado consolidado do processamento. A interface em CLI, ilustrada na Figura 6, funciona como ponto de partida inicial, guiando o usuário pelas etapas do protocolo e exibindo os resultados de cada uma delas.

O funcionamento conjunto desses componentes é baseado em uma troca estruturada de arquivos intermediários. Cada nó executa, de forma independente, uma sequência de scripts locais que implementam as operações do Verificatum. Esses scripts, organizados na pasta `local-scripts` e mostrados na Figura 7, representam o mecanismo utilizado para preparar a chave pública, realizar o embaralhamento e gerar a decifração parcial.

O *frontend* define a ordem de execução desses scripts e verifica a presença dos arquivos gerados em cada etapa, assegurando que os nós avancem de maneira sincronizada.

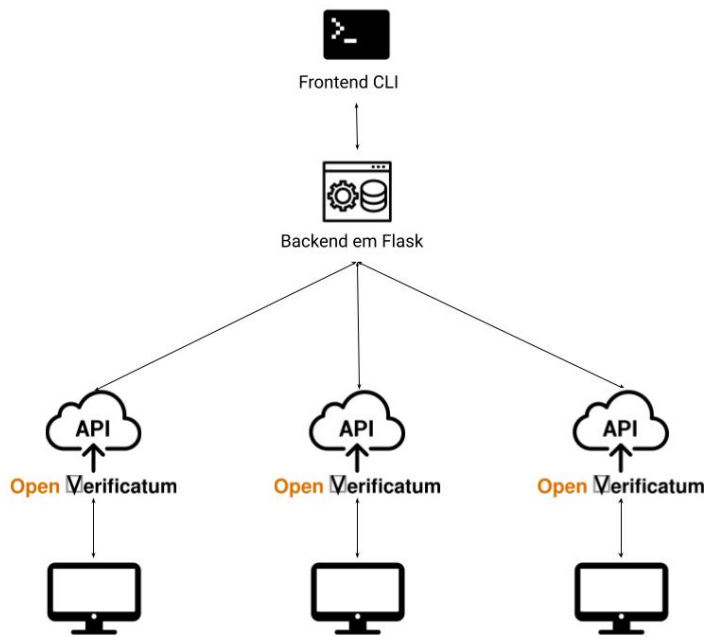


Figura 5 – Esquema geral dos componentes da *mixnet* e sua interação: o *frontend*, *backend* em Flask e os nós executando a API criptográfica

Fonte: Elaboração Própria

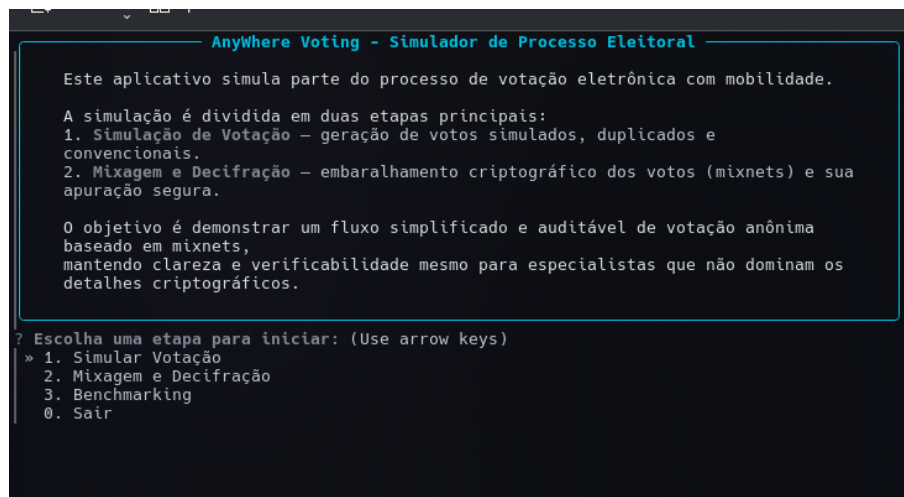


Figura 6 – Tela inicial do frontend em CLI

Essa estrutura possibilita executar o fluxo completo — da preparação criptográfica à decifração final — de forma modular e reproduzível.

As seções seguintes detalham as responsabilidades de cada componente da arquitetura e sua participação no processamento da mixnet.

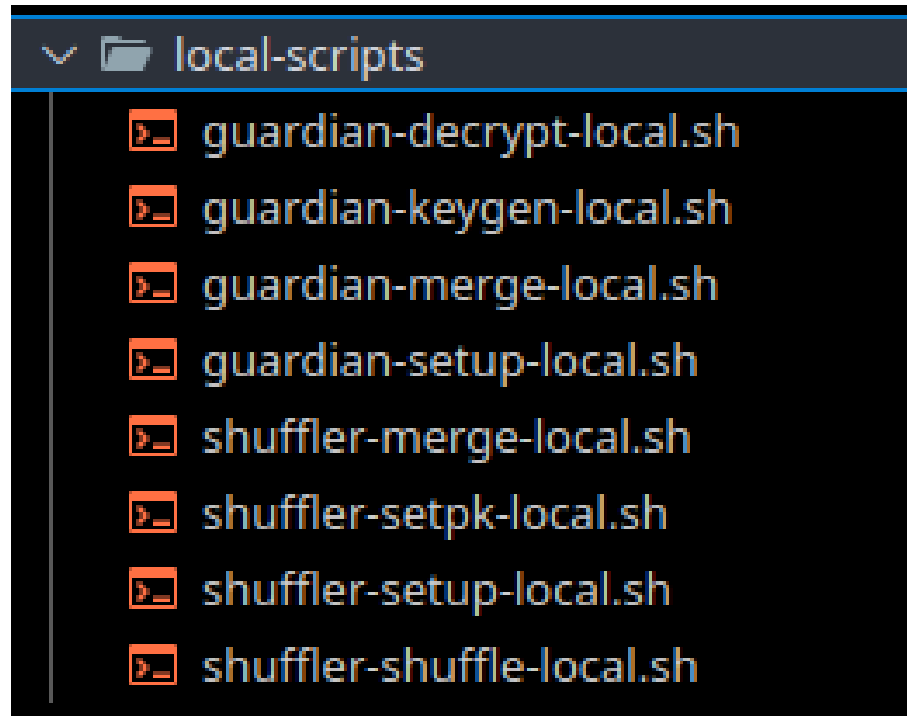


Figura 7 – Conteúdo da pasta `local-scripts` com os scripts de operação

5.3 Módulo 1 — API em Java com Verificatum

A base criptográfica da solução foi implementada a partir do Verificatum, software amplamente utilizado para executar *mixnets* com recifração, provas de conhecimento zero e decifração distribuída. Para integrar essas funcionalidades ao fluxo descrito na Seção 5.1, desenvolveu-se uma API intermediária em Java, responsável por executar localmente as operações necessárias do Verificatum e expor essas operações ao servidor Flask.

Essa API organiza os diretórios internos da execução, aciona os comandos do Verificatum e controla as etapas de *setup*, *shuffle* e decifração parcial. Sua função é tornar o uso da mixnet reproduzível e automatizável, evitando a necessidade de interação direta com os comandos originais do Verificatum. Uma descrição resumida dos endpoints está disponível no Apêndice A.

A arquitetura segue a divisão lógica utilizada pelo Verificatum: uma *Guardian mix network*, composta pelos nós que participam da geração e da decifração distribuída da chave; e uma *Shuffler mix network*, responsável pelo embaralhamento dos votos cifrados e pela produção das provas de correção. Essa separação foi inicialmente motivada por uma tentativa de implementar uma comunicação manual entre os nós por meio de *pen-drives*, em vez do fluxo tradicional via HTTP.

A ideia do uso de *pen-drives* surgiu como alternativa para permitir o controle explícito dos arquivos intermediários produzidos pelo *shuffle*. Essa abordagem teria como vantagem facilitar auditorias manuais e permitir que cada nó executasse isoladamente sua

etapa, transferindo apenas os artefatos necessários para o próximo nó.

Durante os testes, porém, a proposta mostrou-se imprática. O procedimento se tornava lento, exigia intervenção constante do operador e dificultava a automatização do fluxo. Por esses motivos, a comunicação manual foi abandonada e substituída pelas chamadas nativas do Verificatum, que se comunica através de protocolo HTTP na rede local. Ainda assim, a separação entre *Guardian* e *Shuffler* foi mantida, pois ela reflete a divisão nativa do Verificatum e facilita a configuração de ambientes com diferentes quantidades de nós.

Esse módulo constitui o núcleo criptográfico da solução, fornecendo as operações necessárias para o embaralhamento seguro dos votos, produção das provas e decifração parcial cooperativa.

5.4 Módulo 2 — *Frontend* e Simulação de Eleição

O segundo módulo da solução reúne todos os componentes responsáveis pela interação com o operador e pela preparação dos votos processados pela mixnet. Ele é composto por três elementos principais: o *frontend* em linha de comando (CLI), que conduz a execução do fluxo, o simulador de eleição (`MockElection`), que produz os votos cifrados utilizados nas etapas seguintes, e um backend em Flask. A comunicação entre o *frontend* e a API criptográfica é realizada por meio desse servidor Flask, integrado à CLI e apresentado nesta mesma seção.

As próximas subseções descrevem, em nível conceitual, o funcionamento desse módulo e sua integração com a arquitetura geral da solução. Detalhes de implementação, formatos de arquivo e definições de endpoints estão organizados nos apêndices desse trabalho.

5.4.1 Funcionamento do *Frontend* em CLI

O *frontend* em linha de comando funciona como ponto central de operação do sistema. Ele organiza a execução das etapas apresentadas na Seção 5.1, aciona o servidor Flask sempre que uma nova fase do protocolo deve ser iniciada e apresenta ao operador o estado atualizado de cada etapa.

A CLI conduz o processo de maneira sequencial: inicializa os nós guardiões, solicita a geração da chave pública distribuída, envia os votos cifrados para o backend e acompanha as etapas de embaralhamento e decifração. A interface também realiza verificações automáticas, garantindo que os arquivos esperados foram produzidos corretamente e que os nós avançam de forma sincronizada.

O servidor Flask integra-se diretamente ao *frontend* e atua como camada inter-

mediária entre a CLI e a API criptográfica em Java. Embora não execute operações criptográficas, ele organiza o envio de arquivos, prepara os dados necessários para cada fase e encaminha chamadas para os nós da mixnet.

O conjunto formado por CLI e Flask constitui, portanto, a camada operacional do sistema: ele coordena o fluxo de execução, aciona as operações criptográficas quando necessário e fornece ao operador uma visão clara e contínua do andamento do processo.

5.4.2 Simulação e Cifragem de Votos

Para testar o funcionamento completo da mixnet, foi desenvolvido o módulo `MockElection`, responsável por gerar votos sintéticos compatíveis com a chave pública produzida pelos nós guardiões. O módulo simula uma eleição controlada, produzindo plaintexts a partir dos cargos definidos e das quantidades de votos e duplicatas especificadas na configuração da simulação.

Cada plaintext é cifrado com o mesmo esquema utilizado pela mixnet: ElGamal sobre grupo elíptico, conforme a implementação oficial do Verificatum. A chave pública exportada pelos nós guardiões é utilizada diretamente pelo módulo, garantindo que os ciphertexts resultantes tenham o formato esperado pelos nós embaralhadores e possam ser processados sem adaptações intermediárias.

A cifragem é realizada em lote, permitindo produzir grandes quantidades de votos de forma eficiente. Votos duplicados são gerados de maneira determinística, o que facilita a validação do módulo responsável por identificar e remover duplicatas antes do embaralhamento. Ao final do processo, o simulador exporta um arquivo único contendo todos os votos cifrados, que é enviado ao servidor Flask para iniciar o fluxo descrito na Seção 5.1.

Como depende apenas da chave pública e não interfere nas etapas internas da mixnet, o `MockElection` pode ser executado de forma independente. Ele serve como ferramenta para avaliar desempenho, depurar o fluxo e testar diferentes cenários de carga. Detalhes adicionais sobre sua implementação estão disponíveis nos apêndices.

5.5 Módulo 3 - Protótipo Móvel

O aplicativo móvel desenvolvido neste trabalho tem como objetivo ilustrar, de forma concreta, a experiência do eleitor em um cenário de mobilidade na votação. Embora não execute criptografia real, o protótipo implementa um fluxo completo de estados, geração de *payload*, fragmentação em múltiplas partes e renderização de QR Codes de alta densidade. Com isso, reproduz desafios técnicos reais relacionados ao transporte offline de credenciais (tokens) que permitiriam ao eleitor votar em qualquer seção eleitoral habilitada.

Nesse contexto, o aplicativo representa a “camada pré-voto” da solução, responsável pela ativação da mobilidade, geração do token e apresentação do QR Code na ponta do eleitor. O backend Flask, a CLI e a mixnet implementam as etapas de processamento criptográfico e totalização pós-voto, descritas nos módulos anteriores.

5.5.1 Tecnologias e Ambiente de Desenvolvimento

O protótipo foi desenvolvido em React Native com o uso do *framework* Expo. Esse ambiente oferece recarregamento rápido, ferramentas de depuração integradas e grande agilidade para iterar sobre as telas, o que é adequado para refinar a experiência do usuário durante a fase de projeto.

Entretanto, o Expo Go possui limitações relevantes para o cenário deste trabalho: não permite acesso direto a diversas APIs nativas e não gera binários instaláveis (`.apk` ou `.aab`) prontos para distribuição. Para testar o protótipo em condições mais próximas de um aplicativo real, foi necessário utilizar o processo de *prebuild* do Expo, gerando um projeto nativo Android, e posteriormente empacotar a versão final no Android Studio.

A migração para o ambiente nativo expôs diferenças de comportamento em bibliotecas gráficas e de manipulação de QR Codes, exigindo ajustes finos de *assets*, permissões e dimensões de tela. Também foi necessário adaptar parâmetros de renderização para garantir legibilidade dos QR Codes de alta densidade em dispositivos reais.

5.5.2 Arquitetura Interna do Aplicativo

O aplicativo foi estruturado em módulos independentes, organizados em três camadas principais:

- **Interface (UI):** telas e componentes visuais em React Native, com navegação organizada em *stacks* e *bottom tabs*, seguindo um fluxo próximo ao de aplicativos oficiais de serviços eleitorais.
- **Lógica e fluxo:** camada de controle implementada por meio de um contexto global denominado `ElectionContext`, responsável por armazenar e gerenciar o estado da mobilidade ao longo de todo o uso do aplicativo.
- **Simulador de token e QR Code:** responsável por gerar o *payload* do token, simular a presença de biometria cifrada, criar uma assinatura fictícia e dividir o *payload* em múltiplos QR Codes compatíveis com as limitações reais de capacidade.

Essa separação torna o protótipo mais simples de manter e facilita a substituição de partes específicas (por exemplo, a camada de geração de token) sem alterar a lógica de navegação ou a interface.

5.5.3 Fluxo do Eleitor no Aplicativo

Todas as telas do protótipo encontram-se reunidas no Apêndice D. O fluxo do eleitor foi desenhado para reproduzir, em ambiente móvel, as etapas necessárias para ativar a mobilidade de voto e apresentar o token no dia da eleição. As principais telas e ações são:

1. **Login simulado:** o eleitor acessa o aplicativo e carrega um perfil fictício, em fluxo semelhante ao do e-Título.
2. **Informações do eleitor:** exibição dos dados básicos (nome, zona, seção) e de um resumo sobre o que é a mobilidade de voto.
3. **Ativação do voto móvel:** o eleitor opta por aderir à mobilidade, disparando o processo de geração do token.
4. **Seleção do local de votação:** o aplicativo mostra um mapa com seções receptoras fictícias, permitindo ao eleitor escolher onde deseja votar.
5. **Geração e exibição dos QR Codes:** o token é convertido em um conjunto de QR Codes multipartes, que podem ser apresentados à seção receptora no dia da votação.

Esse fluxo não substitui as etapas formais de identificação e habilitação do eleitor, mas ilustra como a mobilidade poderia ser exposta em uma interface voltada ao público geral.

5.5.4 Simulação dos Estados do Processo Eleitoral

Para representar de forma mais fiel o funcionamento da mobilidade, o aplicativo implementa uma máquina de estados interna. O `ElectionContext` mantém esses estados e controla as transições entre as fases do processo. Os principais estados são:

- `BEFORE_REG`: estado inicial, no qual a mobilidade ainda não foi ativada.
- `REG_OPEN`: a adesão à mobilidade foi registrada e o token foi gerado.
- `LOCATION_CHOICE`: o eleitor escolhe a seção receptora onde pretende votar.
- `TOKEN_DISPLAY`: o token é exibido por meio dos QR Codes multipartes.
- `VOTED`: estado final hipotético, que representa o consumo do token no momento do voto.

Esses estados permitem simular todo o ciclo da mobilidade: do registro prévio à apresentação do token. Eles também facilitam a inclusão de regras adicionais, caso o protótipo venha a ser expandido para cenários mais próximos de uma implementação real.

5.5.5 Geração do Token de Mobilidade

A ativação da mobilidade resulta na criação de um objeto `token`, composto por:

- um identificador único `tokenID`;
- uma biometria cifrada simulada;
- o código da cidade do eleitor;
- um conjunto de `metadata`, incluindo horário de geração e parâmetros auxiliares;
- uma assinatura digital fictícia.

Esse *payload* é serializado em formato JSON, produzindo estruturas com tamanho típico entre 10 kB e 15 kB, dependendo dos campos associados à biometria. Esses valores são compatíveis com tamanhos reais encontrados em aplicações que transportam templates biométricos cifrados e assinaturas, e levam diretamente ao problema de capacidade dos QR Codes.

5.5.6 Limitações Reais do QR Code e Motivação para a Divisão em Partes

Mesmo no modo *byte*, QR Codes apresentam limitações significativas de capacidade. A matriz de maior tamanho disponível (versão 40 com correção de erro nível M) suporta aproximadamente 2,9 kB de dados úteis, valor muito inferior ao tamanho típico do token descrito na subseção anterior. Assim, é tecnicamente inviável representar o token completo em um único QR Code sem compressão agressiva ou remoção de campos.

Esse cenário é representativo de soluções reais de mobilidade de voto baseadas em QR Codes, nas quais o token tende a incluir biometria cifrada, identificadores de eleição, parâmetros de uso, metadados de auditoria e assinaturas digitais. O protótipo foi projetado explicitamente para refletir essa limitação, de forma que o usuário visualize um comportamento alinhado aos desafios práticos desse tipo de solução.

Para contornar a limitação de capacidade, o aplicativo divide o token em múltiplos QR Codes. Cada parte é renderizada como um QR Code individual. No aplicativo, o usuário navega por essas partes em um carrossel, com indicação explícita de “Parte *i* de *n*” na interface. Esse esquema é compatível com leitores convencionais de QR Code e representa de forma realista a necessidade, em aplicações offline, de dividir credenciais volumosas em múltiplas telas para viabilizar a captura na ponta receptora.

5.5.7 Desafios Práticos na Renderização dos QR Codes

O uso de QR Codes de alta versão trouxe desafios práticos relevantes durante o desenvolvimento:

- maior uso de memória devido à complexidade da matriz;
- alta densidade de códigos QR traz uma certa incerteza ao sistema: é possível que em alguns dispositivos a leitura seja feita com sucesso, enquanto em outros foi necessário diminuir ainda mais a versão;

Esses fatores aproximam o protótipo dos problemas enfrentados em soluções reais que dependem de QR Codes para transporte de credenciais de alta densidade, reforçando o papel do aplicativo como ferramenta experimental para analisar a viabilidade da mobilidade de voto em cenários com restrição de conectividade.

5.6 Testes e Avaliação

Esta seção trata da etapa de testes e *benchmarking* do projeto, especificamente a frente da API criptográfica. No Apêndice B se encontra um passo-a-passo detalhado de uma execução demonstrativa da fluxo.

5.6.1 Resultados do benchmarking

A seguir, estão dispostos os gráficos resultantes dos processos de *benchmarking*, analisando diferentes métricas como tempo de computação e tamanho de provas geradas para as diversas etapas do processo. A tabela na Figura 6 mostra as especificações da máquina utilizada para o *benchmarking*.

Especificação da Máquina de Benchmarking	
Sistema Operacional	Ubuntu 24.04.3 LTS
Processador	AMD Ryzen™ 5 5500U with Radeon™ Graphics × 12 (4.056 GHz)
Memória	16,0 GiB

Figura 8 – Especificações da máquina utilizada para os processos de *benchmarking*

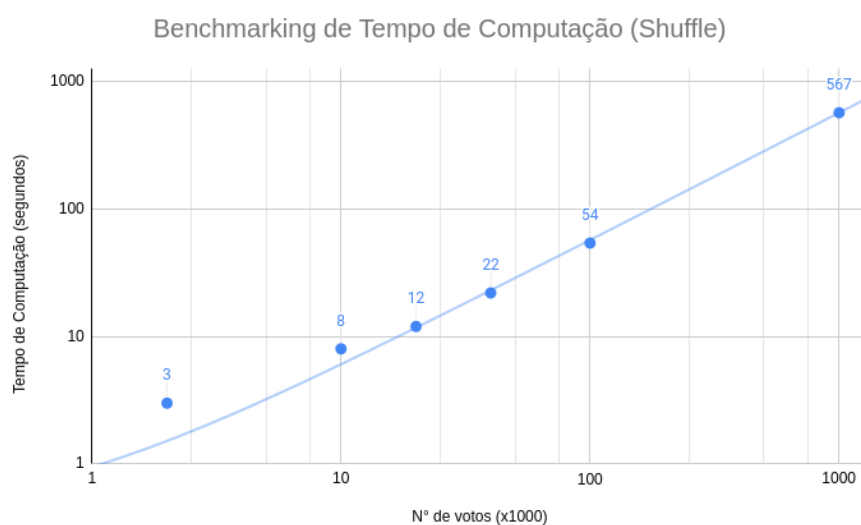


Figura 9 – Gráfico do *benchmarking* de tempo de computação por número de votos na etapa de embaralhamento

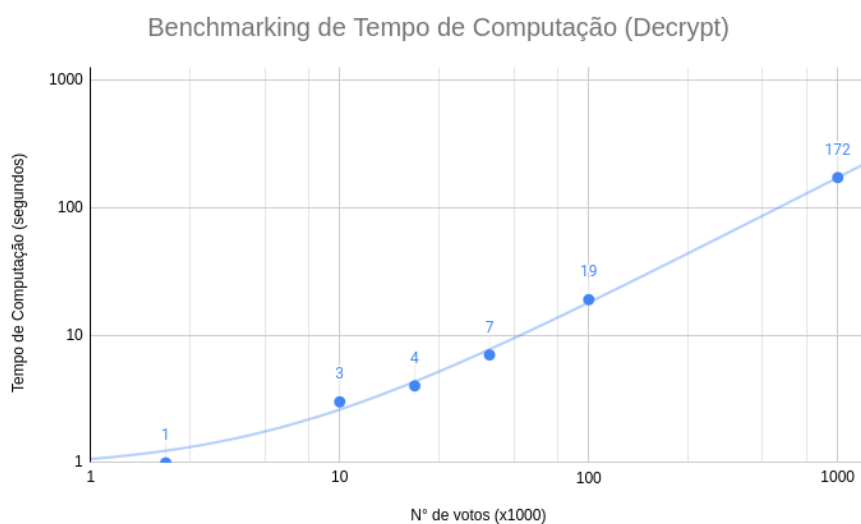


Figura 10 – Gráfico do *benchmarking* de tempo de computação por número de votos na etapa de decifração

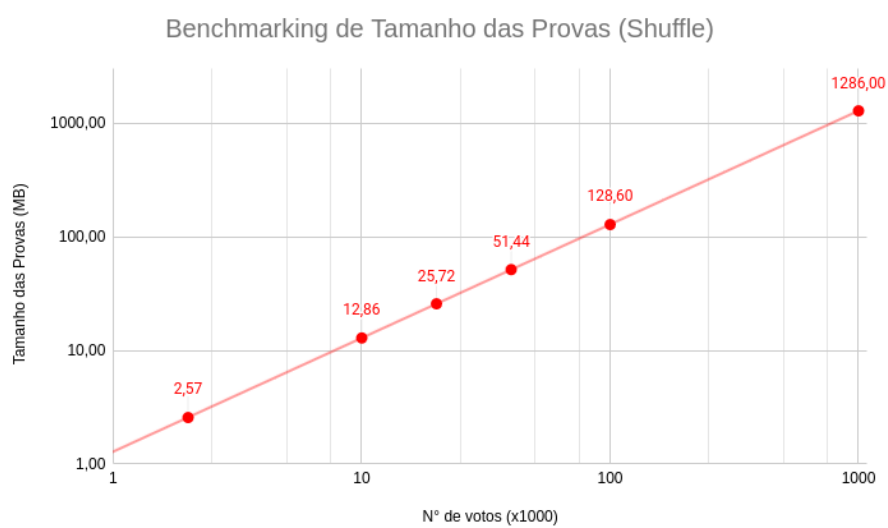


Figura 11 – Gráfico do *benchmarking* de tamanho dos arquivos de provas por número de votos na etapa de embaralhamento

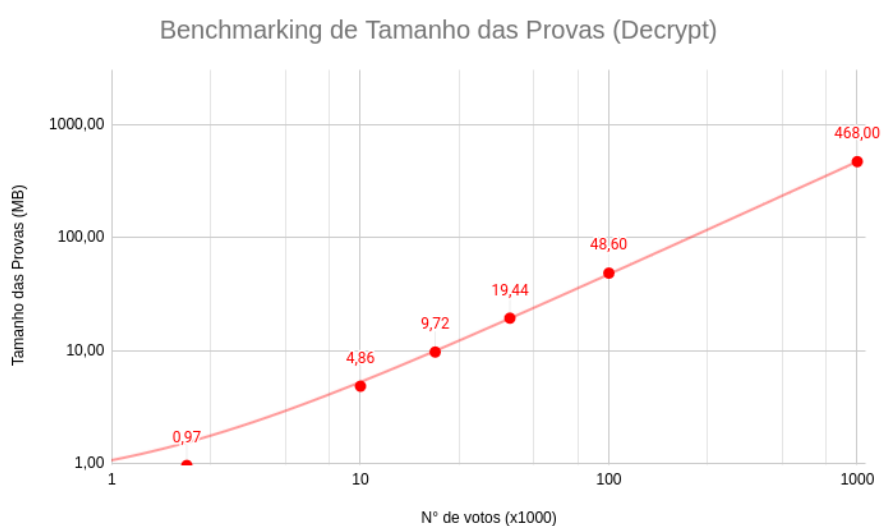


Figura 12 – Gráfico do *benchmarking* de tamanho dos arquivos de provas por número de votos na etapa de decifração

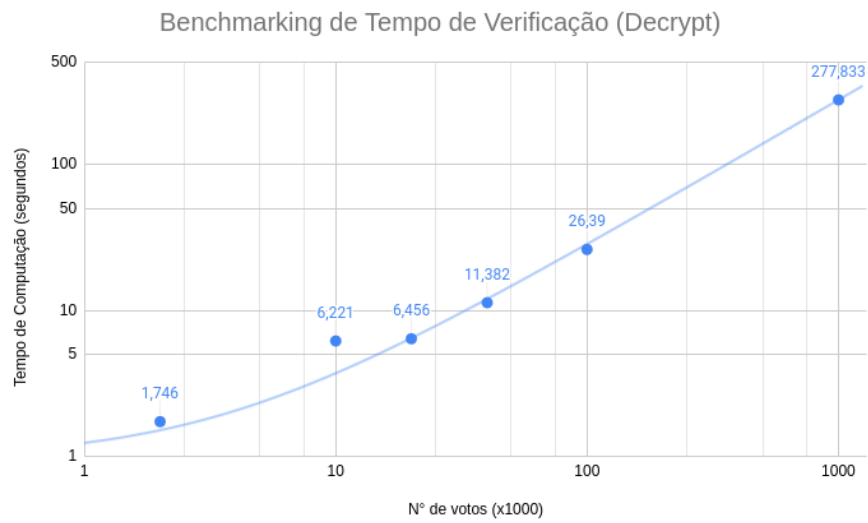


Figura 13 – Gráfico do *benchmarking* de tempo de verificação das provas por número de votos na etapa de decifração

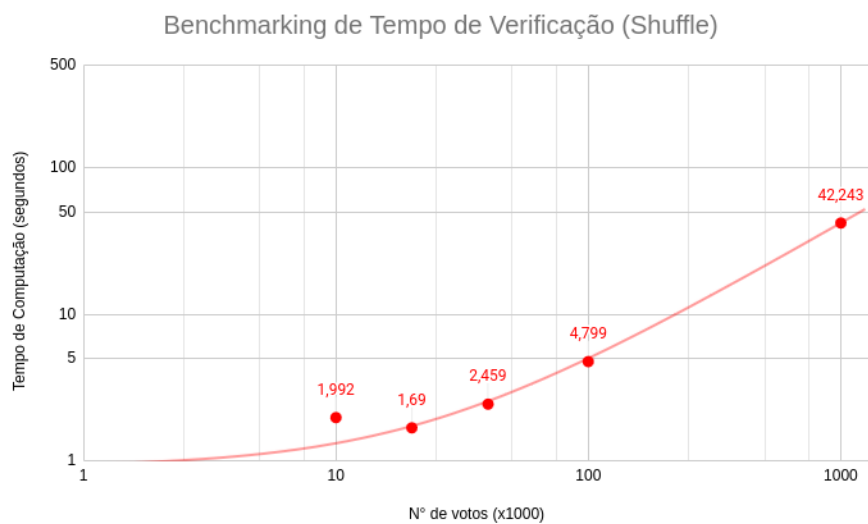


Figura 14 – Gráfico do *benchmarking* de tempo de verificação das provas por número de votos na etapa de embaralhamento

Analisando os dados dispostos nos gráficos de *benchmarking*, podem-se extrair algumas conclusões a respeito do funcionamento da solução da *mix network*. Primeiramente, nota-se que todas as métricas variam, de modo geral, linearmente em relação ao número de votos tratados. Tomando o caso limite testado, do tratamento de 1 milhão de votos duplos, os tempos para cada etapa foram:

- Embaralhamento (*shuffle*): 567 segundos
- Decifração (*decrypt*): 172 segundos
- Verificação do embaralhamento: 278 segundos
- Verificação da decifração: 42 segundos

Somando as etapas, temos um tempo total de processamento de 1059 segundos, ou 17 minutos e 39 segundos. Ou seja, mesmo em um cenário com um número bastante elevado de votos duplos, o tempo de processamento se encontra dentro do limite estabelecido na Seção 3.4 como critério para a viabilidade da solução.

Ademais, quanto aos dados de tamanhos dos arquivos de provas gerados, podemos novamente fazer uma análise pelo caso limite testado, de 1 milhão de votos. Nele, os tamanhos dos arquivos de provas gerados em cada etapa foram:

- Embaralhamento (*shuffle*): 1286 MB
- Decifração (*decrypt*): 468 MB

Somando novamente as etapas, o espaço de armazenamento total requerido para acomodar a geração dos arquivos de provas é de 1754 MB.

6 Considerações Finais

6.1 Conclusões do Projeto de Formatura

Este capítulo apresenta as considerações finais do projeto, retomando seus objetivos, resultados e impactos. Ao longo do desenvolvimento, buscou-se não apenas implementar um protótipo conceitual de um sistema de mobilidade na votação, mas também avaliar, de forma objetiva, a viabilidade técnica de uma solução baseada em *mix networks* para garantir privacidade, auditabilidade e integridade em cenários de voto móvel. Os resultados obtidos — tanto no protótipo do aplicativo quanto na API criptográfica — mostram que a proposta é factível e pode servir de base para evoluções futuras, reforçando o potencial da abordagem adotada. Nas próximas seções, discutem-se o estado final das implementações, as principais contribuições alcançadas e as possibilidades de continuidade deste trabalho.

6.1.1 Estado Final do Aplicativo

Considerando o objetivo inicial do aplicativo — servir como um protótipo de interface para demonstrar, de forma didática, como poderia funcionar a introdução de votos móveis — é possível afirmar que o propósito foi cumprido. O `.apk` gerado ao final foi capaz de executar o fluxo mínimo esperado, apresentando a experiência de um eleitor real em um cenário de mobilidade: simulação dos estados básicos do processo eleitoral dentro do app e exibição de QR Codes contendo dados fictícios compatíveis com o tamanho de um token real.

Ao longo do desenvolvimento, algumas funcionalidades foram deliberadamente mantidas fora do escopo deste protótipo, por se tratarem de extensões naturais da solução completa. Um exemplo é a integração direta do aplicativo com o fluxo de mixagem desenvolvido no restante do projeto. Embora seja tecnicamente possível conectar o token exibido no aplicativo ao `tokenID` utilizado na mixnet, o objetivo aqui foi isolar a “camada pré-voto”, focando na experiência do eleitor, enquanto a parte criptográfica permanece encapsulada na API e na mixnet. Essa integração ponta a ponta é discutida como oportunidade de trabalhos futuros.

De forma semelhante, a implementação da assinatura ECDSA e o controle fino de parâmetros internos do QR Code — como a versão exata da matriz, capacidade e nível de correção de erros — foram identificados como aprimoramentos importantes para uma versão mais próxima de um sistema de produção. No protótipo atual, a biblioteca utilizada (*react-native-qrcode-svg*) já foi suficiente para gerar QR Codes robustos e funcionalmente adequados para testes em dispositivos reais, embora alguns desses parâmetros sejam defi-

nidos automaticamente e não expostos diretamente ao desenvolvedor. Explorar bibliotecas de nível mais baixo, com maior controle sobre esses detalhes, é uma linha clara de evolução técnica do aplicativo.

Por fim, alguns elementos da interface foram simplificados em relação ao que se espera de uma aplicação em produção. O mapa de locais de votação, por exemplo, foi implementado com uma lista fixa de opções pré-definidas, suficiente para ilustrar o fluxo de escolha de seção eleitoral, mas sem a complexidade de uma integração dinâmica com serviços de geolocalização.

Em suma, ainda que existam diversas oportunidades de evolução, o protótipo desenvolvido cumpre o seu papel dentro deste projeto: demonstrar, de forma concreta e utilizável em celulares reais, a experiência de um eleitor em um cenário de voto móvel, conectando visualmente o conceito de token em QR Code à solução criptográfica proposta nas demais camadas.

6.1.2 Estado Final da API

No que diz respeito à API criptográfica e ao fluxo completo de mixagem, os objetivos estabelecidos no início do projeto foram plenamente alcançados. A implementação atual permite executar todo o ciclo de processamento de votos: geração de votos cifrados, execução das etapas de *mixing* com provas de verificação e, por fim, a decifragem e contabilização dos votos de maneira íntegra e auditável. Esse fluxo foi testado tanto em uma única máquina, para facilitar a experimentação rápida durante o desenvolvimento, quanto em máquinas distintas conectadas em rede local, simulando um cenário mais próximo de uma infraestrutura real. Em ambos os casos, o sistema se comportou de forma estável e consistente.

A quantidade de etapas envolvidas no processo reflete deliberadamente uma escolha de design: oferecer visibilidade e controle sobre cada fase da pipeline criptográfica. Enquanto ambientes de demonstração se beneficiam de automatizações para agilizar o uso, cenários mais próximos de uma operação real exigem que cada etapa seja explícita e auditável. Assim, a estrutura atual busca equilibrar esses dois propósitos, sem perder o rigor das garantias criptográficas proporcionadas pela biblioteca *Verificatum* e pela orquestração desenvolvida.

Além disso, os *benchmarks* realizados demonstraram que a solução é viável do ponto de vista de desempenho, atendendo aos critérios definidos na Seção 3.4. As medições de tempo nas principais fases — cifragem, mixagem e decifragem — fornecem indicadores concretos de que a abordagem baseada em *mix networks* pode ser aplicada a cenários reais, especialmente considerando possíveis otimizações futuras.

Portanto, o estado final da API evidencia um protótipo funcional, capaz de executar

um fluxo completo de votação anônima, verificável e resistente à vinculação entre eleitor e voto. A arquitetura desenvolvida estabelece uma base sólida para trabalhos futuros, que podem explorar simplificações do fluxo, melhorias de paralelização ou integração com outros componentes do sistema.

6.2 Contribuições

O principal resultado deste trabalho, na frente da API criptográfica, foi demonstrar na prática a viabilidade de uma solução de mobilidade apoiada em *mix networks*, conforme sugerido no artigo “How to Implement Anywhere Voting: A Case Study of Brazil” (KIMURA; ARAÚJO; SIMPLICIO, 2025). Implementou-se um fluxo completo, desde a geração de votos cifrados até a mixagem, verificação e decifragem final, atendendo aos critérios de viabilidade definidos na Seção 3.4 e avaliados na Seção 5.6. Esse resultado constitui uma evidência concreta — e inédita no contexto brasileiro — de que é possível integrar mobilidade eleitoral com anonimato forte e auditabilidade criptográfica, utilizando componentes reais e executando o protocolo em máquinas distintas conectadas em rede.

Além disso, o trabalho produziu *benchmarks* detalhados das principais etapas da pipeline, oferecendo parâmetros objetivos para estimar desempenho, escalabilidade e custos computacionais de soluções análogas. Esses dados servem como referência para pesquisadores e engenheiros interessados em avaliar alternativas de implementação baseadas em *mix networks*, contribuindo diretamente para o avanço da discussão sobre voto seguro em cenários de mobilidade.

No desenvolvimento do aplicativo móvel, a contribuição central foi tornar tangível a experiência do voto móvel para um eleitor real. O protótipo apresenta um fluxo de uso simples, educativo e alinhado ao que seria esperado em um sistema desse tipo, permitindo visualizar de maneira clara o papel do token de votação. Um destaque importante foi a implementação dos QR Codes com tamanhos equivalentes aos encontrados em um contexto eleitoral real. Por exemplo, se um template biométrico genuíno ocuparia cerca de 12 kB em ASCII, os QR Codes gerados pelo aplicativo também carregam aproximadamente esse volume de dados, ainda que fictícios.

Essa construção permitiu avaliar, diretamente em celulares reais, como seria manipular tokens grandes, quais versões de QR Code proporcionam melhor leitura, e como a densidade da matriz impacta a experiência prática do eleitor. Essa dimensão empírica, acrescenta valor ao trabalho por tornar visíveis aspectos de usabilidade que normalmente ficam ocultos em estudos puramente teóricos.

Em conjunto, as duas frentes do projeto — a solução criptográfica funcional e o protótipo móvel — formam uma demonstração completa e coerente do potencial do voto móvel aliado a técnicas modernas de anonimização, oferecendo uma base sólida para

explorações técnicas, acadêmicas e institucionais futuras.

6.3 Perspectivas de Continuidade

Os resultados obtidos ao longo do projeto abriram diversas possibilidades de desdobramentos futuros, tanto do ponto de vista técnico quanto conceitual. Uma evolução natural consiste em integrar, de maneira mais direta, as camadas atualmente separadas da solução: o aplicativo móvel, o servidor Flask e o fluxo de mixnet. Embora cada componente tenha sido desenvolvido de forma independente para permitir clareza, modularidade e testes isolados, uma integração ponta a ponta possibilitaria visualizar todo o percurso do voto móvel em uma única demonstração unificada. Tal conexão ampliaria o caráter didático da solução e permitiria explorar questões práticas de interoperabilidade entre as etapas de geração, transporte, mixagem e contabilização de votos.

No âmbito do aplicativo, há espaço significativo para expandir sua dinâmica e torná-lo mais representativo de um sistema real. A criação de um backend dedicado permitiria substituir estruturas estáticas por elementos carregados dinamicamente, como o mapa de locais de votação, informações do eleitor e atualizações do estado do processo eleitoral. Essa interação tornaria o protótipo mais flexível e permitiria avaliar aspectos de usabilidade que só emergem em cenários mais próximos do uso real.

Outra frente promissora envolve o aperfeiçoamento dos QR Codes. A adoção de uma biblioteca que ofereça controle mais granular sobre os parâmetros internos — tais como versão, capacidade, densidade e níveis de correção de erro — permitiria testar diferentes configurações da matriz e compreender melhor a relação entre tamanho do token, legibilidade e experiência do usuário. Além disso, expor esses parâmetros na interface do aplicativo pode enriquecer futuras apresentações e facilitar discussões técnicas sobre a adequação de cada variante do QR Code para cenários de votação móvel.

Por fim, no que diz respeito à arquitetura criptográfica, trabalhos futuros podem investigar diferentes formas de otimização do fluxo de mixagem, incluindo paralelização, automatizações opcionais, novas formas de orquestração e integração com mecanismos de monitoramento ou auditoria. Tais aprimoramentos podem contribuir para tornar a solução ainda mais eficiente e adaptável a diferentes escalas de eleição.

Em conjunto, essas perspectivas representam caminhos claros de evolução que podem aprofundar, ampliar e consolidar a proposta apresentada neste projeto, reforçando seu papel como base para pesquisas e protótipos mais avançados na área de mobilidade eleitoral segura.

Referências

- AB, V. *User Manual for the Verificatum Mix-Net: Vmn version 3.1.0*. Suécia, 2022. Manual técnico. Disponível em: <<https://www.verificatum.org>>. Acesso em: 10 dez. 2025. Citado na página 20.
- COMINETTI, E. L. *Improving the Brazilian electronic voting system through the use of end-to-end verifiability*. 174 p. Tese (Tese (Doutorado)) — Universidade de São Paulo, Escola Politécnica, São Paulo, 2024. Citado 5 vezes nas páginas 9, 12, 13, 15 e 19.
- DENSO WAVE Incorporated. *QR Code: Version Information*. 2025. Disponível em: <<https://www.qrcode.com/en/about/version.html>>. Acesso em: 02 dez. 2025. Citado na página 15.
- DESMEDT, Y. G. Threshold cryptography. *ETT – Engineering, Technology & Teaching*, v. 5, n. 4, p. 449–457, jul./ago. 1994. Citado na página 13.
- HASAN, J. Overview and applications of zero knowledge proof (zkp). *International Journal of Computer Science and Network*, v. 8, n. 5, p. 1–44, oct. 2019. ISSN 2277-5420. IJSCSN – International Journal of Computer Science and Network. Disponível em: <<http://www.ijcsn.org>>. Acesso em: 10 dez. 2025. Citado na página 14.
- KIMURA, L.; ARAÚJO, R.; SIMPLICIO, M. How to implement anywhere voting: A case study of brazil: 10th international joint conference, e-vote-id 2025, nancy, france, october 1–3, 2025, proceedings. In: DUENAS-CID, D. et al. (Ed.). *Electronic Voting*. Cham: Springer, 2025, (Lecture Notes in Computer Science, v. 16028). p. 124–140. Disponível em: <https://link.springer.com/chapter/10.1007/978-3-032-05036-6_8>. Acesso em: 10 dez. 2025. Citado 5 vezes nas páginas 9, 10, 17, 18 e 45.
- KNAPP, J.; ZERATSKY, J.; KOWITZ, B. *Sprint: How to Solve Big Problems and Test New Ideas in Just Five Days*. New York: Simon & Schuster, 2016. Best-seller do Google Ventures. Citado na página 21.
- TIWARI, S. An introduction to qr code technology. In: *2016 International Conference on Information Technology (ICIT)*. Bhubaneswar, Índia: IEEE, 2016. p. 39–44. Citado na página 15.
- Tribunal Superior Eleitoral. *Voto em trânsito - Perguntas e respostas*. Brasília, DF, Brasil, 2022. Disponível em: <<https://www.tse.jus.br/eleicoes/eleicoes-2022/voto-em-transito-perguntas-e-respostas/>>. Acesso em: 09 dez. 2025. Citado 2 vezes nas páginas 9 e 11.
- Tribunal Superior Eleitoral. *Estatísticas eleitorais*. Brasília, DF, Brasil, 2025. Disponível em: <<https://sig.tse.jus.br/ords/dwapr/r/seai/sig-eleicao/home>>. Acesso em: 09 dez. 2025. Citado na página 22.
- Tribunal Superior Eleitoral. *Transferência de título eleitoral*. Brasília, DF, Brasil, 2025. Disponível em: <<https://www.tre-sp.jus.br/servicos-eleitorais/carta-de-servicos/transferencia-de-titulo-de-eleitor>>. Acesso em: 09 dez. 2025. Citado na página 9.

U.S. Election Assistance Commission. *End to End (E2E) Protocol Evaluation Process*. Washington, DC, United States, 2023. Disponível em: <<https://www.eac.gov/voting-equipment/end-end-e2e-protocol-evaluation-process>>. Acesso em: 02 abr. 2023. Citado na página 19.

VERIFICATUM. *Open Verificatum Project*. 2024. Site oficial do Verificatum. Disponível em: <www.verificatum.org>. Acesso em: 16 abr 2025. Citado 2 vezes nas páginas 14 e 19.

XAVIER, B. *Datafolha: 34% dos brasileiros dizem que não teriam ido votar se não fosse obrigatório*. São Paulo, SP, Brasil, 2024. Disponível em: <<https://www1.folha.uol.com.br/poder/2024/10/datafolha-34-dos-brasileiros-dizem-que-nao-teriam-ido-votar-se-nao-fosse-obrigatorio.shtml>>. Acesso em: 10 dez. 2025. Citado na página 9.

Apêndices

APÊNDICE A – Endpoints da API em Java

- `/guardian/setup`: Chamado pelo *front-end* da API, inicia o setup do nó 1 da *mix network Guardian*
- `/guardian/setup-local`: Chamado por *script shell*, inicia o setup de um nó específico na máquina local
- `/guardian/merge-local`: Chamado por *script shell*, executa o *merge* dos arquivos *protInfo0x.xml* de todos os nós que se encontram na pasta do nó da máquina local
- `/guardian/keygen`: Chamado pelo *front-end* da API, inicia o processo de geração de chave pública no nó 1
- `/guardian/keygen-local-async`: Chamado por *script shell*, inicia o processo de geração de chave pública no nó da máquina local
- `/guardian/decrypt`: Chamado pelo *front-end* da API, inicia a decifração no nó 1
- `/guardian/decrypt-local-async`: Chamado por *script shell*, inicia a decifração no nó da máquina local
- `/guardian/public-key`: Chamado pelo *front-end*, emite a chave pública gerada pela *mix network Guardian*

Endpoints da *mix network Shuffler*:

- `/shuffler/setup`: Chamado pelo *front-end*, inicia o setup do nó 1 da *mix network Shuffler*
- `/shuffler/setup-local`: Chamado por *script shell*, inicia o setup do nó instanciado na máquina local
- `/shuffler/merge-local`: Chamado por *script shell*, executa o *merge* dos arquivos *protInfo0x.xml* gerados pelo setup de todos os nós, que se encontram na pasta do nó local
- `/shuffler/setpk-local`: Chamado por *script shell*, configura a chave pública recebida da *mix network Guardian* para ser utilizada na recifração realizada pela *mix network Shuffler*
- `/shuffler/receive-ciphertexts`: Chamado pelo *Flask*, recebe os *ciphertexts* (votos cifrados) gerados, que serão embaralhados pela *mix network*

- `/shuffler/shuffle`: Chamado pelo *front-end*, inicia o processo de embaralhamento no nó 1
- `/shuffler/shuffle-local`: Chamado por *script shell*, inicia o embaralhamento em nó local
- `/shuffler/shuffled-ciphertexts`: Chamado pelo *front-end*, fornece os votos já embaralhados e recifrados
- `/shuffler/log`: Chamado pelo *front-end*, fornece o *log* do processo de embaralhamento realizado

APÊNDICE B – Execução Completa da Demonstração Local (Passo a Passo)

Para efeitos de demonstração, a execução a seguir será realizada com todos os nós instanciados na mesma máquina e, antes de iniciar o processo, é um pré-requisito ter instalada na máquina a ser utilizada a biblioteca Verificatum. Seguem os comandos a serem executados a partir do terminal para o funcionamento do fluxo:

1. Instalação e inicialização da API

- a) `git clone https://github.com/jonasgdm/verificatum-api.git`
- b) `cd verificatum-api`
- c) `mvn clean install`
- d) `mvn spring-boot:run`

2. Inicialização do servidor Flask

- a) `cd flask_backend/`
- b) `source .venv/bin/activate`
- c) `python3 app.py`

3. Inicialização do front-end

- a) `cd cli-front/`
- b) `source .venv/bin/activate`
- c) `python3 main.py`

4. Execução completa dos protocolos

- a) `cd local-scripts/`
- b) Realizar o **setup** pelo frontend (figuras 15, 16 e 17)

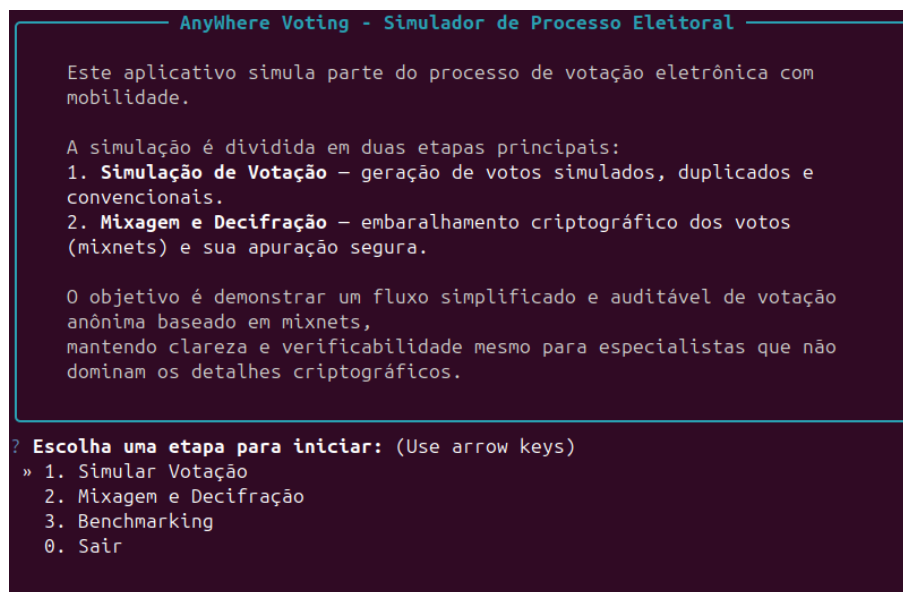


Figura 15 – Tela inicial, selecionando opção 1: Simular Votação



Figura 16 – Tela de início da simulação de votação

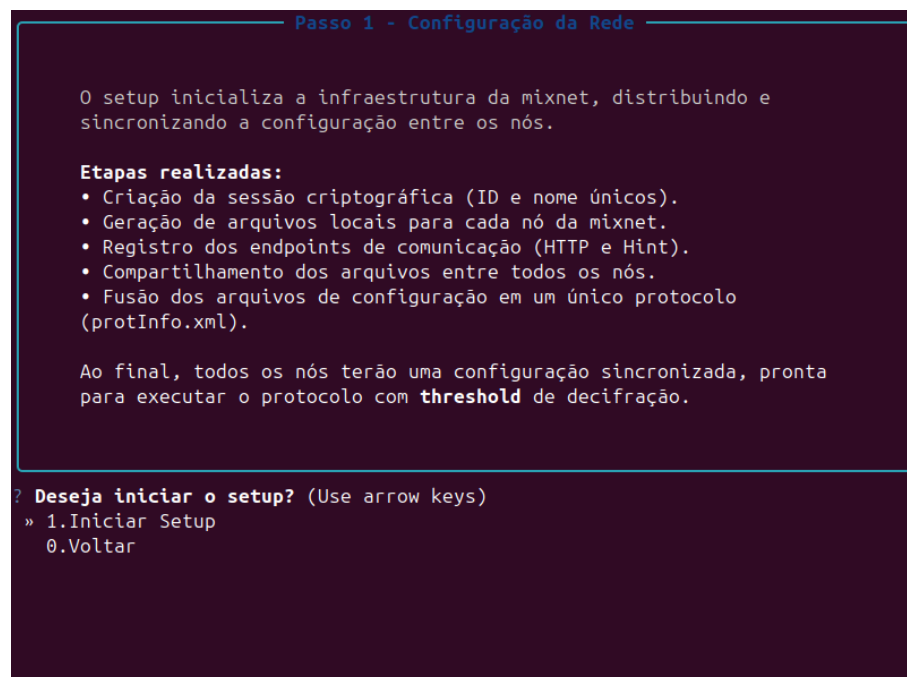


Figura 17 – Tela de início do setup da mix network Guardian

- c) `./local-scripts/guardian-setup-local.sh` (executar nos servidores 2 e 3, como mostra a Figura 18)

```
jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-setup-local.sh
Insira o número do Nó Guardiã local:
2
Insira o número total de nós guardiões: 3
Insira o número threshold de nós: 2
Insira o ID da sessão: GuardianSession
Insira o nome da eleição: Guardian_Election
{"status": "Setup local complete (server 2)"}jonas@localhost:~/verificatum-api/local-scripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-setup-local.sh
Insira o número do Nó Guardiã local:
3
Insira o número total de nós guardiões: 3
Insira o número threshold de nós: 2
Insira o ID da sessão: GuardianSession
Insira o nome da eleição: Guardian_Election
{"status": "Setup local complete (server 3)"}jonas@localhost:~/verificatum-api/local-scripts$
```

Figura 18 – Execução dos scripts locais de setup nos nós 2 e 3 da mix network Guardian

- d) Transferir os arquivos `protInfo0x` gerados para a pasta de **todos** os servidores (em todos os computadores), como mostra a Figura 19

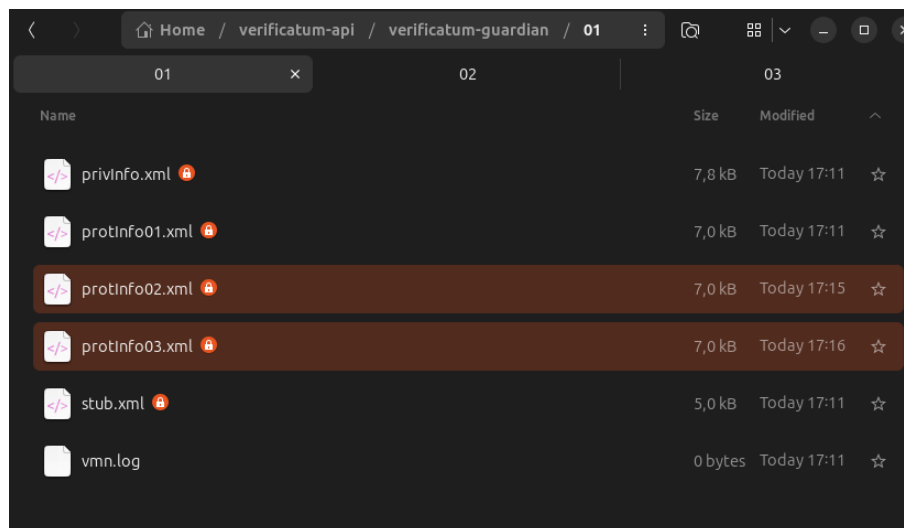


Figura 19 – Pasta do nó 1 da mix network Guardian após transferências de arquivos `protInfo` entre os nós

- e) `./local-scripts/guardian-merge-local.sh` (executar nos servidores 1, 2 e 3, como mostra a Figura 20)

```

jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-merge-local.sh
Insira o número do Nó Guardiã local:
1
Insira o número total de nós guardiões: 3
{"status":"Merge local complete (server 1)"}jonas@localhost:~/verificatum-api/local-scripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-merge-local.sh
Insira o número do Nó Guardiã local:
2
Insira o número total de nós guardiões: 3
{"status":"Merge local complete (server 2)"}jonas@localhost:~/verificatum-api/local-scripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-merge-local.sh
Insira o número do Nó Guardiã local:
3
Insira o número total de nós guardiões: 3
{"status":"Merge local complete (server 3)"}jonas@localhost:~/verificatum-api/local-scripts$

```

Figura 20 – Execução dos scripts locais de merge nos nós 1, 2 e 3 da mix network Guardian

f) Realizar o **keygen** pelo frontend, como mostra a Figura 21

Passo 2 - Geração de Chave Pública

Nesta etapa, cada servidor da mixnet gera sua parcela da chave privada, e juntos constroem uma chave pública compartilhada, que será usada para cifrar os votos.

Etapas realizadas:

- Cada nó executa o algoritmo de geração de chaves (vmn -keygen).
- As parcelas privadas permanecem locais e secretas.
- A chave pública resultante é compartilhada e idêntica entre os nós.
- Essa chave será utilizada para cifragem dos votos simulados.

Importante: A decifração dos votos só poderá ocorrer com a colaboração de um número mínimo de servidores, definido pelo parâmetro **threshold** configurado no setup.

? Deseja gerar a chave pública da eleição? (Use arrow keys)

» 1. Iniciar KeyGen

0. Voltar

Figura 21 – Tela de início da etapa de geração de chave pública

g) `./local-scripts/guardian-keygen-local.sh` (servidores 2 e 3, como mostra a Figura 22, com a chave resultante na Figura 23)

```

jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-keygen-local.sh
Insira o número do Nó Guardiã local:
2
{"status":"keygen started (server 2)"}jonas@localhost:~/verificatum-api/local-scripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-keygen-local.sh
Insira o número do Nó Guardiã local:
3
{"status":"keygen started (server 3)"}jonas@localhost:~/verificatum-api/local-scripts$

```

Figura 22 – Execução dos scripts locais de geração de chave nos nós 2 e 3 da mix network Guardian

```
✓ Keygen concluído com sucesso!

Chave Pública da Eleição

0000000002000000000201000000020636f6d2e766572696669636174756d2e61726974
686d2e4543715047726f75700100000005502d3235360000000020000000002010000
0021006b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c296
0100000021004fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837
bf51f50000000002010000002100b6c7c2852aad4e03b5bac0db10a2811a5603bfb5c4
3f155f0a2b0658fad10c45010000002100c30a452d2294936cca7a3c774e051234be28
c8d12b6609b6b424189600be4d20

[CONTINUAR]
```

Figura 23 – Chave pública resultante do processo de geração na mix network Guardian

- h) Realizar a configuração do número de votos (e duplicatas) da eleição pelo frontend, como mostra a Figura 24

```
Passo 3 - Simulação de Votos

Esta etapa gera votos simulados que são cifrados com a chave pública
da mixnet. Após a geração, os votos são processados para remover
duplicações. Apenas a parte cifrada (escolhas por cargo) é enviada à
mixnet para o embaralhamento.

Geração e Cifragem → Remoção de Duplicados → Envio à Mixnet

Configure os parâmetros em electionConfig.json

? Quantos AnyVotes deseja gerar? 1000
? Quantos votos duplicados deseja gerar? 200
```

Figura 24 – Tela de configuração dos votos válidos e duplicatas a serem gerados na eleição da demonstração

- i) Realizar o **shuffle setup** pelo frontend, como nas figuras 25 e 26

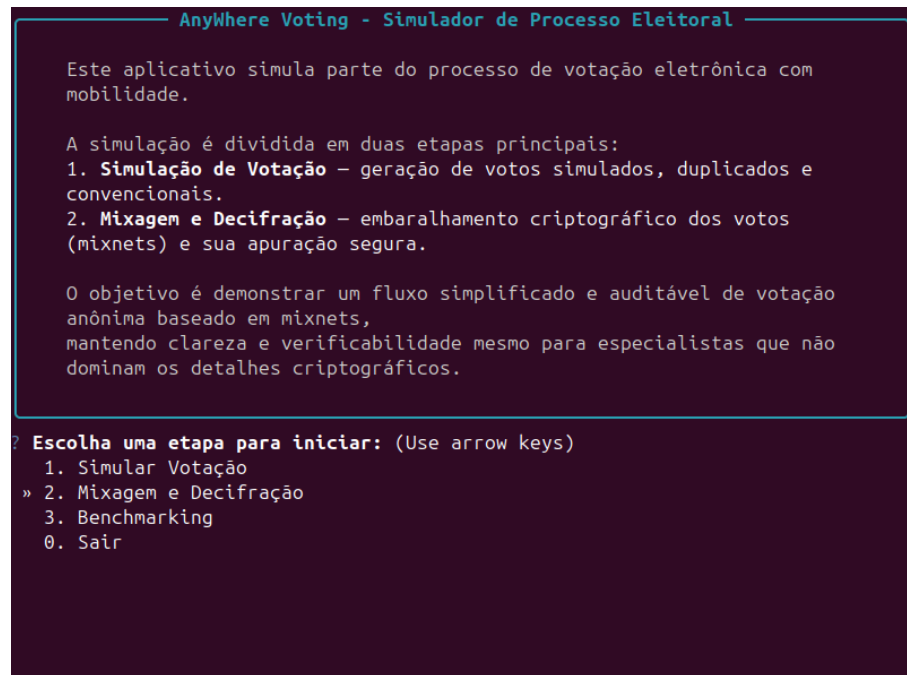


Figura 25 – Tela inicial, selecionando opção 2: Mixagem e Decifração

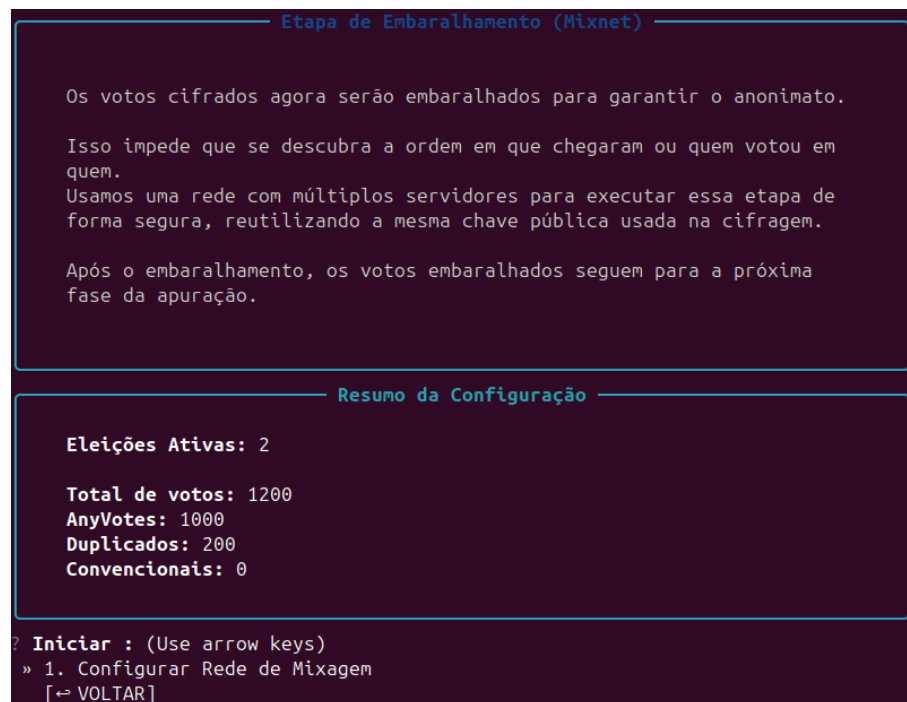


Figura 26 – Tela de início da configuração da mix network Shuffler

- j) `./local-scripts/shuffle-setup-local.sh` (servidores 2 e 3, como na Figura 27)

```

jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-setup-local.sh
Insira o número do Nó de Mixagem local:
2
Insira o número total de nós de mixagem: 3
Insira o número threshold de nós: 2
Insira o ID da sessão: ShuffleSession
Insira o nome da eleição: ShufflerNet
{"status":"Setup local complete (server 2)"}jonas@localhost:~/verificatum-a
cal-scripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-setup-local.sh
Insira o número do Nó de Mixagem local:
3
Insira o número total de nós de mixagem: 3
Insira o número threshold de nós: 2
Insira o ID da sessão: ShuffleSession
Insira o nome da eleição: ShufflerNet
{"status":"Setup local complete (server 3)"}jonas@localhost:~/verificatum-a

```

Figura 27 – Execução de scripts locais de setup da mix network Shuffler nos nós 2 e 3

- k) Transferir os arquivos `protInfo0x` para **todos** os servidores, como mostra a Figura 28

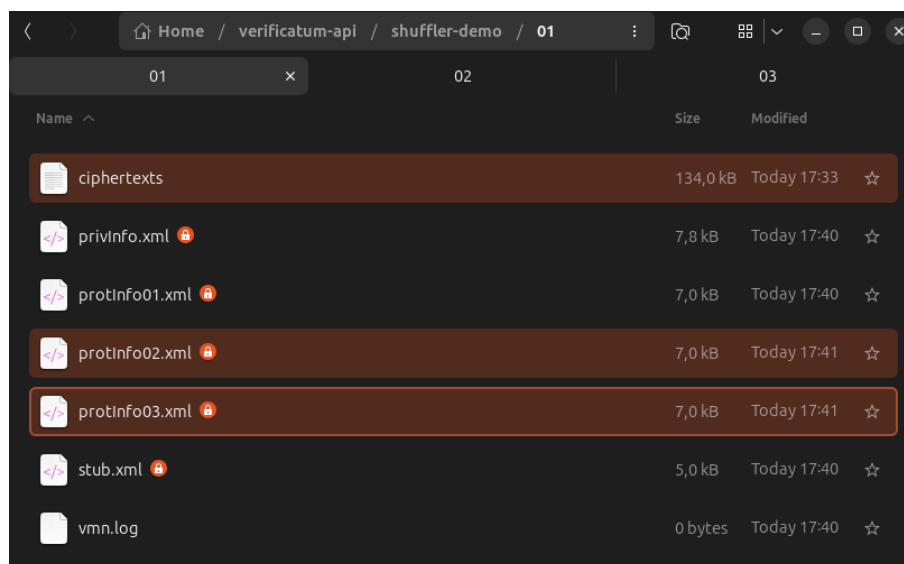


Figura 28 – Pasta do nó 1 da mix network Shuffler após transferências de arquivos entre nós

- l) `./local-scripts/shuffler-merge-local.sh` (Figura 29)

```

jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-merge-local.sh
Insira o número do Nó de Mixagem local:
1
Insira o número total de nós de mixagem: 3
{"status":"Merge local complete (server 1)"}jonas@localhost:~/verificatum-api/local-scripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-merge-local.sh
Insira o número do Nó de Mixagem local:
2
Insira o número total de nós de mixagem: 3
{"status":"Merge local complete (server 2)"}jonas@localhost:~/verificatum-api/local-scripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-merge-local.sh
Insira o número do Nó de Mixagem local:
3
Insira o número total de nós de mixagem: 3
{"status":"Merge local complete (server 3)"}jonas@localhost:~/verificatum-api/local-scripts$

```

Figura 29 – Execução de scripts locais de merge da mix network Shuffler nos nós 1, 2 e 3

m) `./local-scripts/shuffler-setpk-local.sh` (Figura 30)

```

jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-setpk-local.sh
Insira o número do Nó de Mixagem local:
1
{"status":"Set public key local complete (server 1)"}jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-setpk-local.sh
Insira o número do Nó de Mixagem local:
2
{"status":"Set public key local complete (server 2)"}jonas@localhost:~/verificatum-api/local-scripts$ ./shuffler-setpk-local.sh
Insira o número do Nó de Mixagem local:
3
{"status":"Set public key local complete (server 3)"}jonas@localhost:~/verificatum-api/local-scripts$

```

Figura 30 – Execução de scripts locais de definição da chave pública da mix network Shuffler nos nós 1, 2 e 3

n) Iniciar o **shuffle** pelo frontend (o servidor 1 fica suspenso e entrega os **ciphertexts** à pasta do nó 1), como mostra a Figura 31

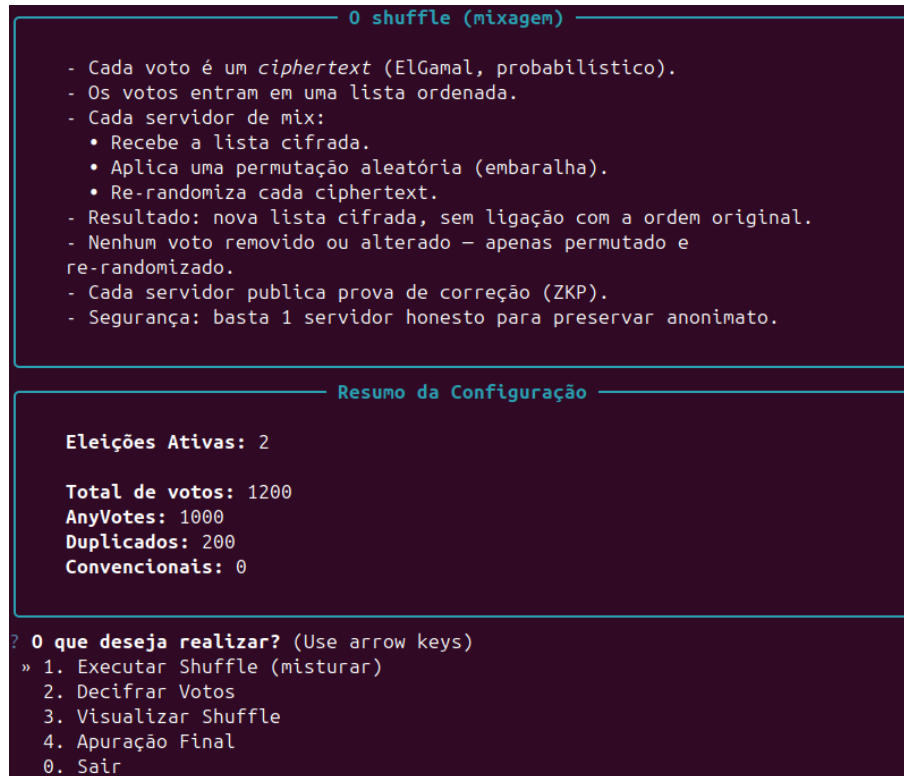


Figura 31 – Tela inicial da mix network Shuffler, selecionando opção 1: Executar Shuffle (misturar)

- o) Transferir os *ciphertexts* do servidor 01 para **todos** os outros servidores (figura 21, novamente)
- p) `./local-scripts/shuffler-shuffle-local.sh` (servidores 2 e 3, como na Figura 32)

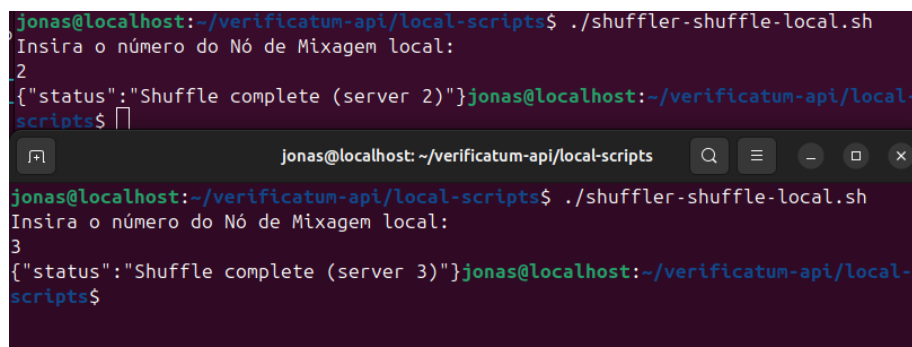


Figura 32 – Execução de scripts locais de mistura da mix network Shuffler nos nós 2 e 3

- q) Voltar à aba do front-end que estava suspensa e pressionar *Enter* → a execução continua e termina com sucesso (figuras 33 e 34)


```

0 shuffle (mixagem)

- Cada voto é um ciphertext (ElGamal, probabilístico).
- Os votos entram em uma lista ordenada.
- Cada servidor de mix:
  • Recebe a lista cifrada.
  • Aplica uma permutação aleatória (embaralha).
  • Re-randomiza cada ciphertext.
- Resultado: nova lista cifrada, sem ligação com a ordem original.
- Nenhum voto removido ou alterado – apenas permutado e re-randomizado.
- Cada servidor publica prova de correção (ZKP).
- Segurança: basta 1 servidor honesto para preservar anonimato.

Resumo da Configuração

Eleições Ativas: 2

Total de votos: 1200
AnyVotes: 1000
Duplicados: 200
Convencionais: 0

? 0 que deseja realizar? 1. Executar Shuffle (misturar)
:: Processando...
Erro ao realizar o shuffle.

Voltando...
:: Processando...

```

Figura 33 – Inicialmente o front-end apresenta mensagem de erro

```

  • Re-randomiza cada ciphertext.
- Resultado: nova lista cifrada, sem ligação com a ordem original.
- Nenhum voto removido ou alterado – apenas permutado e re-randomizado.
- Cada servidor publica prova de correção (ZKP).
- Segurança: basta 1 servidor honesto para preservar anonimato.

Resumo da Configuração

Eleições Ativas: 2

Total de votos: 1200
AnyVotes: 1000
Duplicados: 200
Convencionais: 0

? 0 que deseja realizar? 1. Executar Shuffle (misturar)
:: Processando...
Erro ao realizar o shuffle.

Voltando...
:: Processando...
:: Processando...

✓ Shuffle Conclu.
Vá para 'Visualizar Shuffle' e visualize o embaralhamento

Pressione Enter para continuar...

```

Figura 34 – Após pressionar *Enter* na tela com erro, mensagem de sucesso aparece, finalizando etapa de mistura

- r) Mover o arquivo `/shuffler-demo/0x/shuffled` para `/verificatum-guardian/0x/` (figuras 35 e 36)

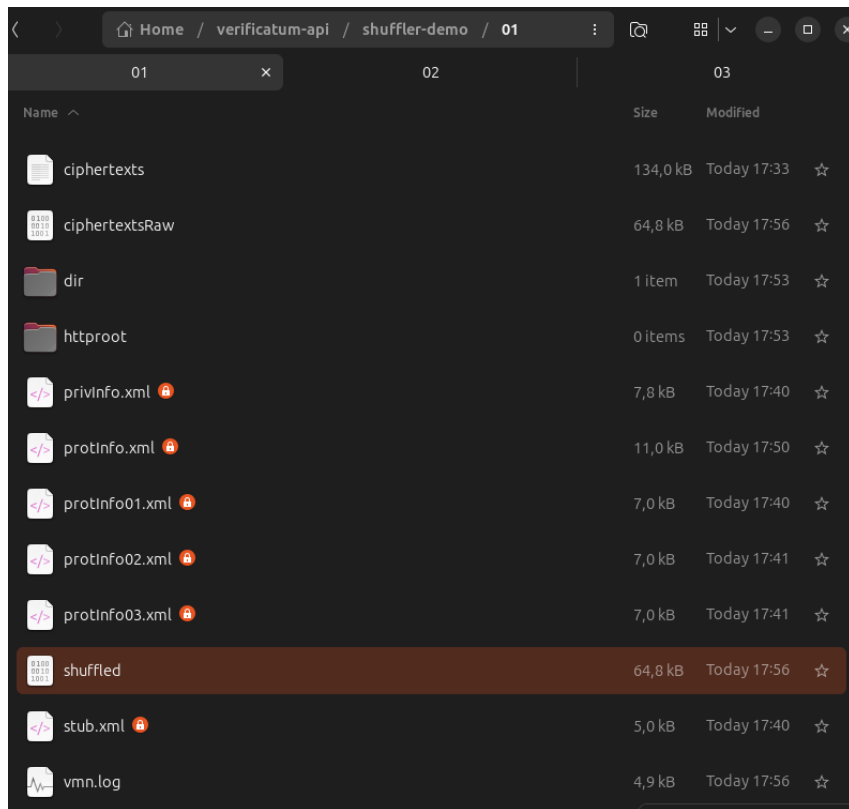


Figura 35 – Pasta do nó 1 da mix network Shuffler após mistura (shuffle), copiando o arquivo gerado *shuffled*

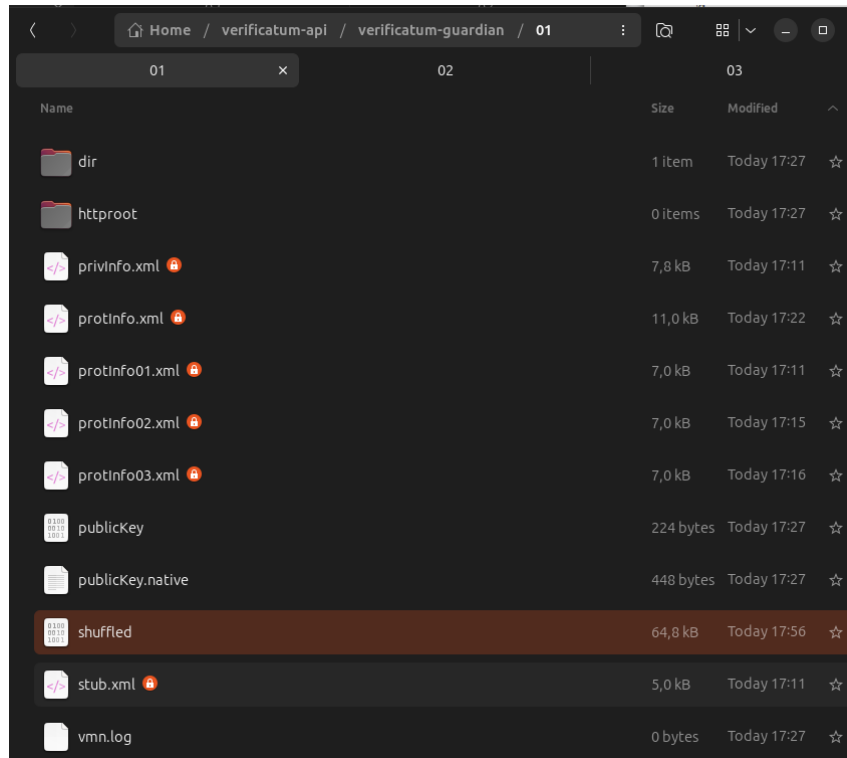


Figura 36 – Pasta do nó 1 da mix network Guardian após transferência do arquivo *shuffled* da mix network Shuffler para os nós da mix network Guardian

s) Realizar o **decrypt** pelo frontend (Figura 37)

```

0 shuffle (mixagem)

- Cada voto é um ciphertext (ElGamal, probabilístico).
- Os votos entram em uma lista ordenada.
- Cada servidor de mix:
  • Recebe a lista cifrada.
  • Aplica uma permutação aleatória (embaralha).
  • Re-randomiza cada ciphertext.
- Resultado: nova lista cifrada, sem ligação com a ordem original.
- Nenhum voto removido ou alterado – apenas permutado e re-randomizado.
- Cada servidor publica prova de correção (ZKP).
- Segurança: basta 1 servidor honesto para preservar anonimato.

Resumo da Configuração

Eleições Ativas: 2

Total de votos: 1200
AnyVotes: 1000
Duplicados: 200
Convencionais: 0

? 0 que deseja realizar? (Use arrow keys)
  1. Executar Shuffle (misturar)
  » 2. Decifrar Votos
  3. Visualizar Shuffle
  4. Apuração Final
  0. Sair

```

Figura 37 – Tela de início da etapa de decifração, selecionando opção 2: Decifrar Votos

- t) `./local-scripts/guardian-decrypt-local.sh` (servidores 2 e 3, como na Figura 38, com a conclusão da etapa na Figura 39)

```

jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-decrypt-local.sh
Insira o número do Nó Guardiã local:
2
{"status":"decrypt started (server 2)"}jonas@localhost:~/verificatum-api/local
cripts$
jonas@localhost:~/verificatum-api/local-scripts$ ./guardian-decrypt-local.sh
Insira o número do Nó Guardiã local:
3
{"status":"decrypt started (server 3)"}jonas@localhost:~/verificatum-api/local

```

Figura 38 – Execução de scripts locais de decifração da mix network Guardian nos nós 2 e 3

```

- Os votos entram em uma lista ordenada.
- Cada servidor de mix:
  • Recebe a lista cifrada.
  • Aplica uma permutação aleatória (embaralha).
  • Re-randomiza cada ciphertext.
- Resultado: nova lista cifrada, sem ligação com a ordem original.
- Nenhum voto removido ou alterado – apenas permutado e re-randomizado.
- Cada servidor publica prova de correção (ZKP).
- Segurança: basta 1 servidor honesto para preservar anonimato.

Resumo da Configuração

Eleições Ativas: 2

Total de votos: 1200
AnyVotes: 1000
Duplicados: 200
Convencionais: 0

? 0 que deseja realizar? 2. Decifrar Votos
: Processando...

✓ Decifração concluída.
O arquivo foi salvo em 'decrypted/decrypted.native'.
Agora vá para Apuração Final para ver os resultados.

Pressione Enter para continuar...

```

Figura 39 – Tela de sucesso da etapa de decifração

- u) Realizar a **verificação** das provas geradas pelos processos de decifração e mistura pelo terminal (Figura 40)

```

jonas@localhost:~/verificatum-api/verificatum-guardian/01$ time vmnv -decrypt protInfo.xml dir/nizkp/default/

real    0m0,558s
user    0m1,145s
sys     0m0,088s
jonas@localhost:~/verificatum-api/verificatum-guardian/01$ cd ../../shuffler-demo/01/
jonas@localhost:~/verificatum-api/shuffler-demo/01$ time vmnv -shuffle protInfo.xml dir/nizkp/default/

real    0m1,160s
user    0m2,243s
sys     0m0,154s
jonas@localhost:~/verificatum-api/shuffler-demo/01$

```

Figura 40 – Execução de comandos de verificação das provas das etapas de decifração e de mistura

Observação: Todos os comandos devem ser executados nos terminais correspondentes a cada máquina/servidor, exceto as ações explicitamente indicadas como “pelo front-end” (interface web da API, acoplada ao nó 1 das mix networks Guardian e Shuffle).

APÊNDICE C – Código Responsável pela Cifração

```
// encryptor/encrypt_core.js
require("./min-vjsc-1.1.1.js");
const verificatum = global.verificatum;

// ——— ESTADO PERSISTENTE (preparado no init) ———
let state = {
  ready: false,
  keyPGroup: null,
  wpk: null,
  eg_master: null,          // ElGamal com PRG mestre
  prg_master: null,         // PRG mestre (semeado 1x)
};

function initWithPublicKey(publicKeyJsonString) {
  const pk = JSON.parse(publicKeyJsonString);
  const bt = verificatum.eio.ByteTree.readByteTreeFromByteArray(pk);

  const keyPGroup = verificatum.arithm.ECqPGroup.fromByteTree(bt.value);
  const fullPublicKey = new verificatum.arithm.PPGroup(keyPGroup, 2).to

  // PRG mestre semeado UMA vez via RandomDevice
  const prg_master = new verificatum.crypto.SHA256PRG();
  const seed = new verificatum.crypto.RandomDevice().getBytes(verificatum);
  prg_master.setSeed(seed);

  const eg_master = new verificatum.crypto.ElGamal(true, keyPGroup, prg_master);
  const WIDTH = 1;
  const wpk = eg_master.widePublicKey(fullPublicKey, WIDTH);

  state = { ready: true, keyPGroup, wpk, eg_master, prg_master };
}
```

```

// Cifra UMA mensagem usando objetos já prontos
function encryptOnce(plaintext) {
  // Deriva uma seed fresca do PRG mestre (não chama RandomDevice de novo)
  const derived = state.prg_master.getBytes(verificatum.crypto.SHA256PRG.seed);
  const prg_local = new verificatum.crypto.SHA256PRG();
  prg_local.setSeed(derived);

  // Usa um ElGamal leve com o PRG derivado (mantém segurança e randomização)
  const eg_local = new verificatum.crypto.ElGamal(true, state.keyPGroup, prg_local);

  const ascii = [...plaintext].map(c => c.charCodeAt(0));
  const encoded = state.wpk.pGroup.project(1).encode(ascii);
  const encrypted = eg_local.encrypt(state.wpk, encoded);
  const btBytes = encrypted.toByteTree().toByteArray();
  return Buffer.from(btBytes).toString("hex");
  // const e0 = encrypted.values[0].toByteTree().toByteArray();
  // const e1 = encrypted.values[1].toByteTree().toByteArray();
  // const u8 = new Uint8Array(e0.length + e1.length);
  // u8.set(e0, 0);
  // u8.set(e1, e0.length);
  // return Buffer.from(u8).toString("hex"); // já retorna HEX
}

module.exports = { initWithPublicKey, encryptOnce };

\end{list}

```


APÊNDICE D – Telas do Aplicativo

Protótipo

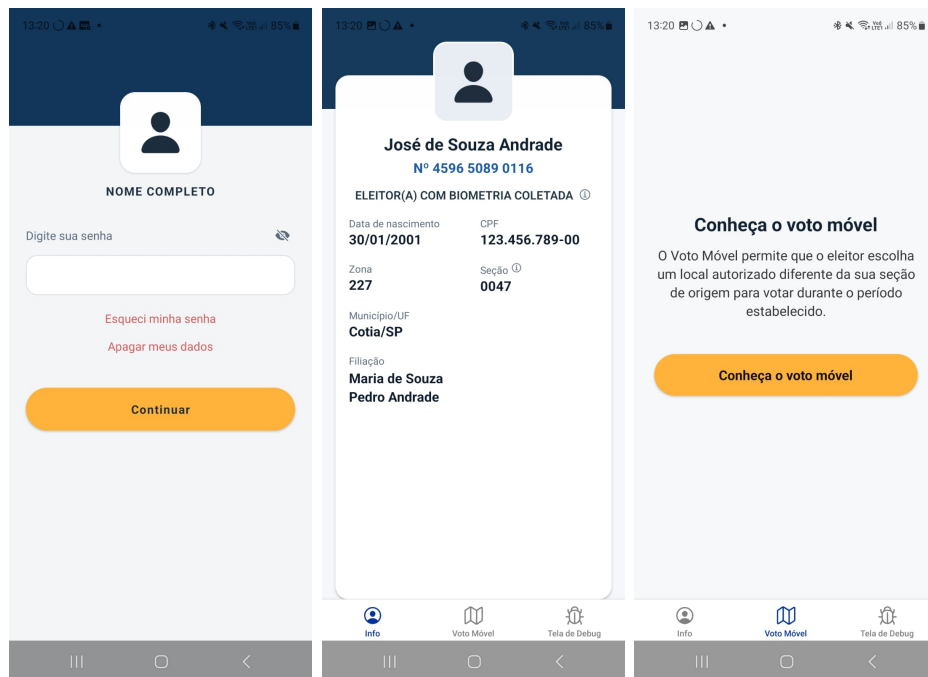


Figura 41 – Telas de login, perfil e inicial do aplicativo.

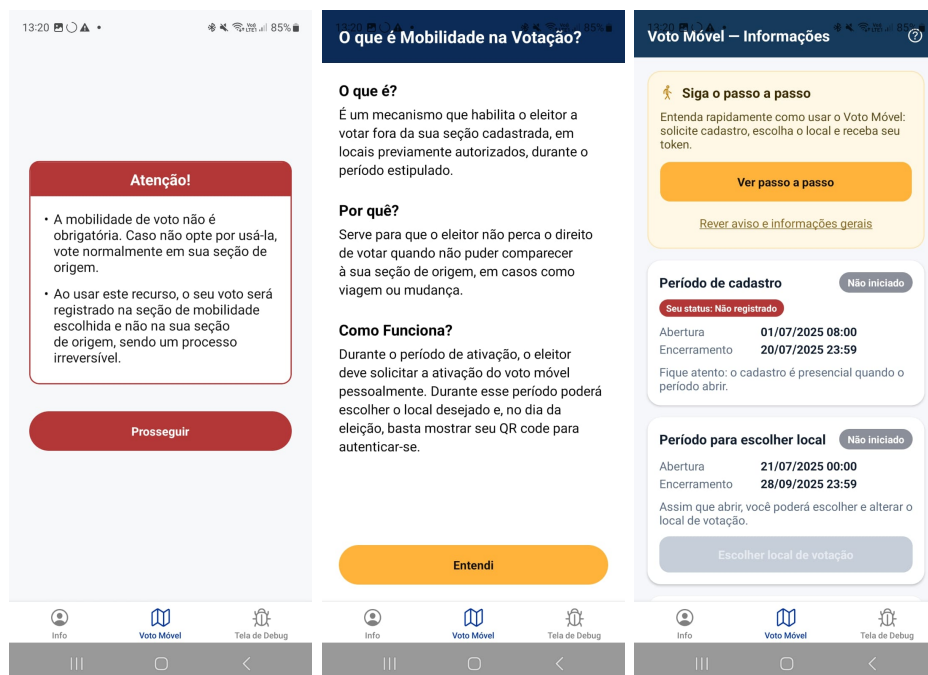


Figura 42 – Telas de aviso, FAQ e menu do aplicativo.

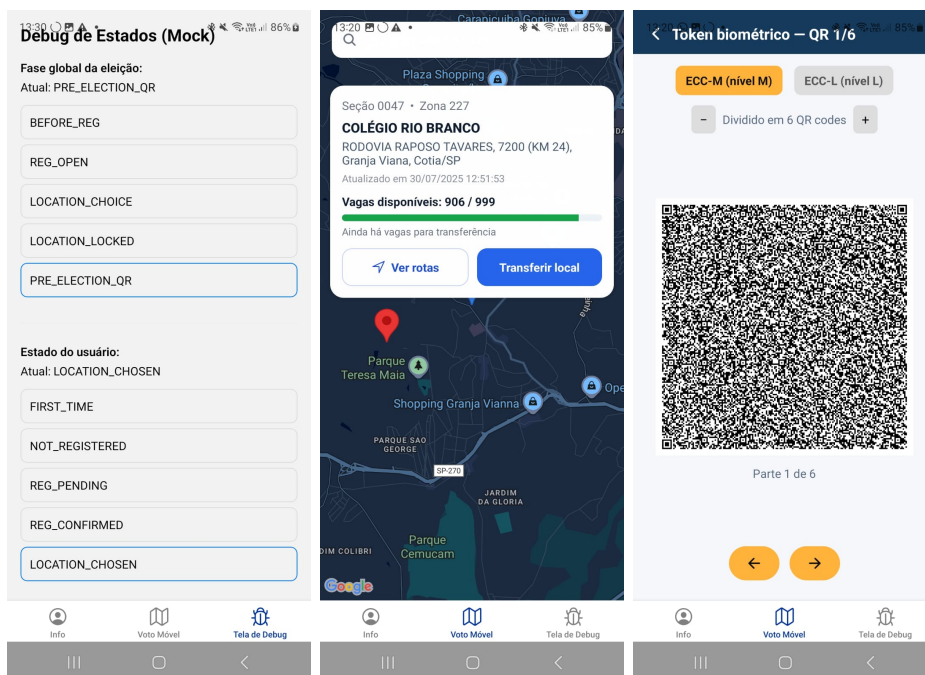


Figura 43 – Telas de depuração, mapa e QR Code do aplicativo.

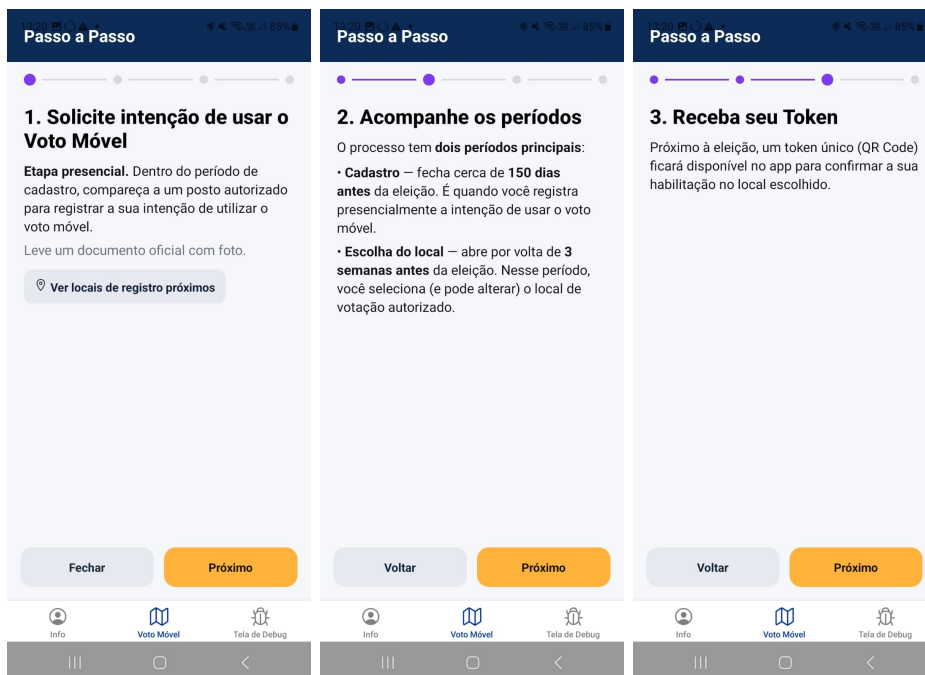


Figura 44 – Fluxo de uso do aplicativo (Passos 1, 2 e 3).

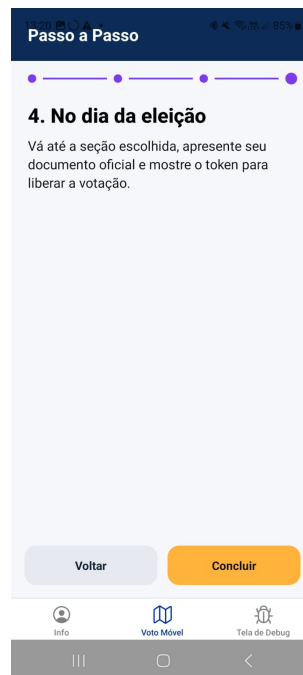


Figura 45 – Fluxo de uso do aplicativo (Passo 4).